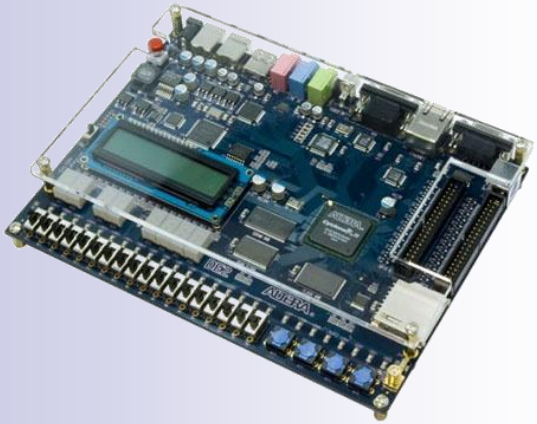
Logic Systems and Processors
cz:Logické systémy a procesory

Lecturer: Richard Šusta

susta@fel.cvut.cz, richard@susta.cz
+420 2 2435 **7359**



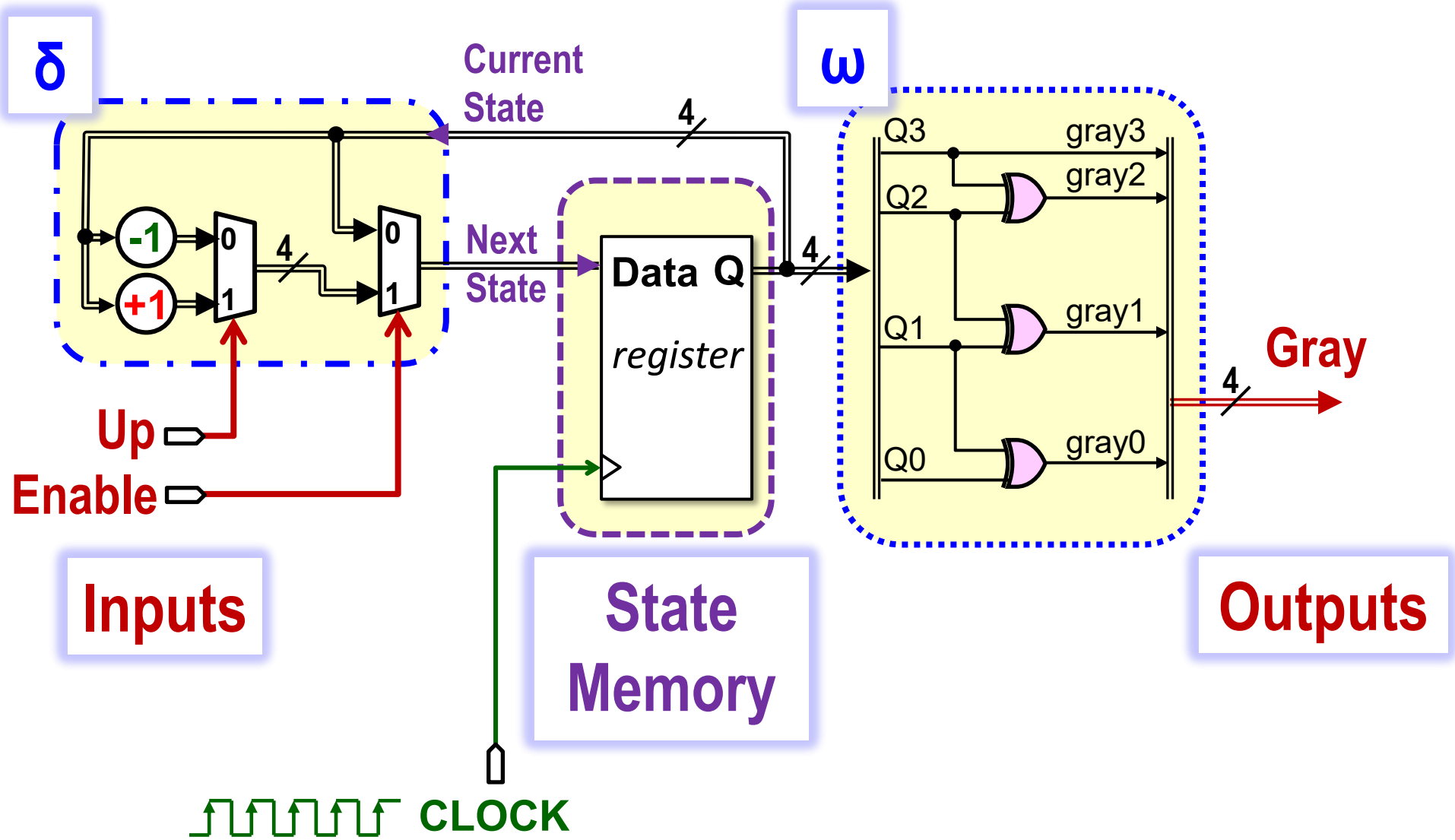
Version: 1.0

CTU-FEE in Prague, CR – subject BE5B35LSP

Finite State Machines

Picture from http://ithare.com/

Gray Code Counter as FSM

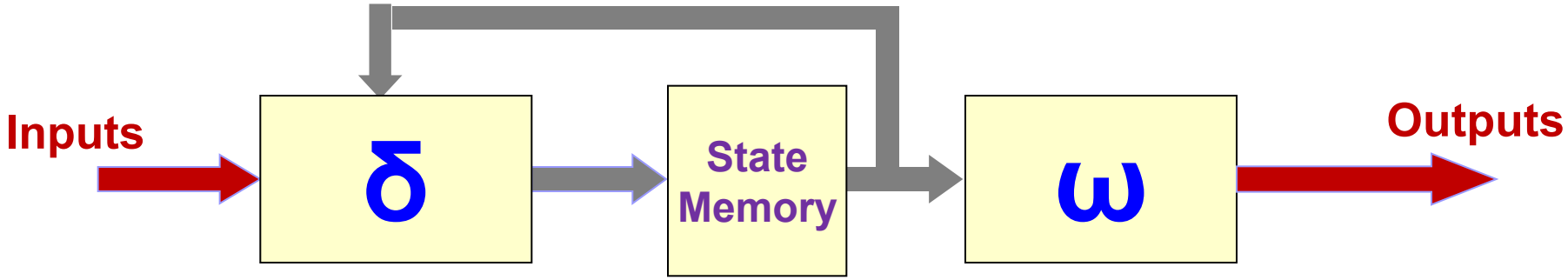


Mealy and Moore Finite State Machine

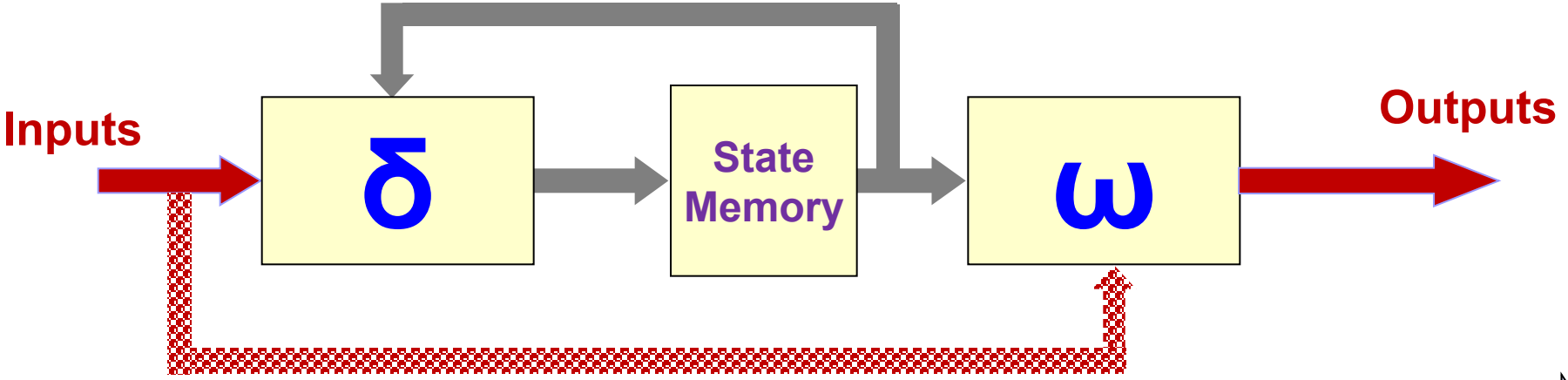
δ - next state combinational circuit

ω - output circuit, combinational/sequential

Moore's automaton



Mealy's automat



Mealy's machine is a year older :-)

- **G. H. Mealy**, "A Method for Synthesizing Sequential Circuits," *Bell Systems Tech. J.*, vol. 34, pp. 1045-1079, September 1955.
- **E. F. Moore**, "**Gedanken-Experiments** on Sequential Machines," *Annals of Mathematical Studies*, no. 34, pp. 129-153, 1956, Princeton Univ. Press, NJ.

Language note:

Gedanken *= plural noun in German*

Gedanke = idea, thought. In English, it is used with "Gedanken-experiment" to describe an approach more conceptual rather than an actual experiment. For example, Albert Einstein often referred to his ideas of hypothetical experiments as such. Moore uses this term to emphasize that some FSMs lead to models that are too complex for any physical construction.

Systems with Internal States

Nonlinear system

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x},\mathbf{u})$$

$$\mathbf{y} = \mathbf{h}(\mathbf{x},\mathbf{u})$$

x - vector of n state variables
u - vector of m inputs
y - vector p of outputs

can be linearized in a working point

$$\frac{d\mathbf{x}}{dt} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$$

$$\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u}$$

Matrix A: $n \times n$, **B:** $n \times \mathrm{m}$
C: $p \times n$, **D:** $p \times \mathrm{m}$

or converted to a discrete form

$$\mathbf{x}[\tau+1] = \mathbf{f}(\mathbf{x}[\tau],\mathbf{u}[\tau])$$

$$\mathbf{y}[\tau] = \mathbf{h}(\mathbf{x}[\tau],\mathbf{u}[\tau])$$

The values of continuous variables are sampled with period T.
The choice of T is critical!

FSM
Deterministic, binary inputs and outputs

$$\mathbf{s}[k+1] = \delta(\mathbf{s}[k],\mathbf{u}[k])$$

$$\mathbf{y}[k] = \omega(\mathbf{s}[k],\mathbf{u}[k])$$

Binary state, inputs, and outputs at time k.
The FSM frequency can vary over a wide range.

FSM

FSM uses a concept similar to the control theory of continuous systems.

- δ - transient function calculates the next state from the current state and inputs and corresponds to the A and B matrices or the f(x,u) function of a discrete model.
- The ω -output function of the FSM generates outputs based on the current states, similarly to the C and D matrices or h(x, u) function of a discrete model.
- The D matrix will speed up responses, but also increase the difficulties of controlling. Natural systems often have D=0, or h(x), i.e., they are analogous to Moore's FSM.

Terminology

English: **Automaton,** plural **Automata**

– *technical English derives its plural essentially from the Latin-style declensions!*

Even if the plural "automatons" is grammatically correct, its usage in science or academic papers is considered inappropriate.

In technical English, we distinguish:

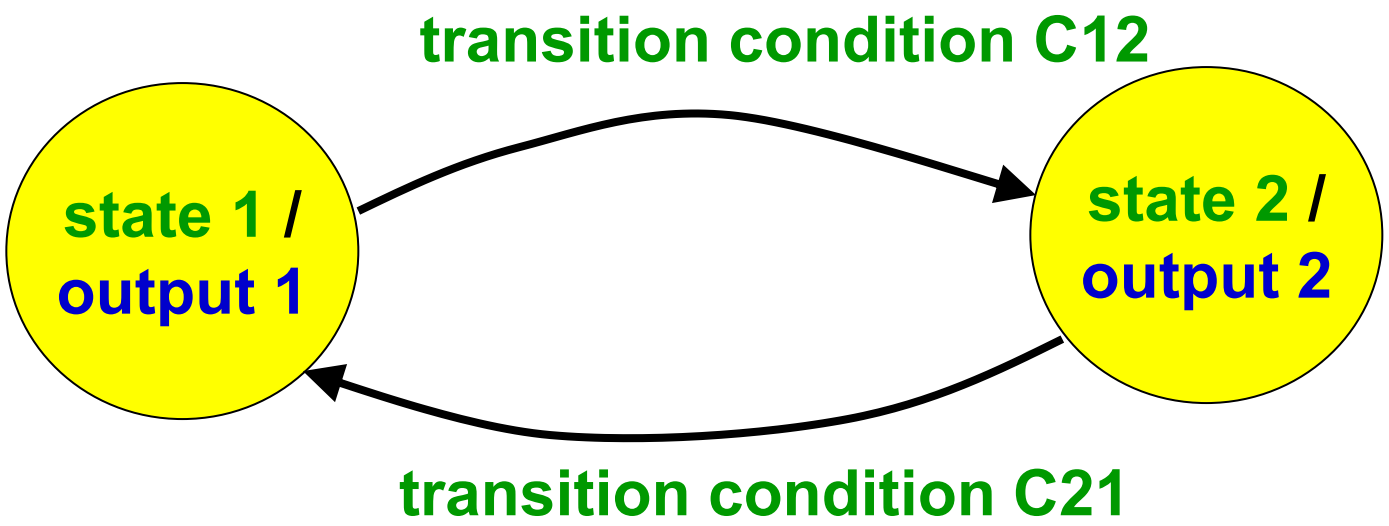
- „**Automaton**“, from Greek self-moving "αὐτόματον" (pron. *automaton*), it means any self-operating machine.
- **FSM "Finite State Machine"** specifies a deterministic automaton with only binary inputs and outputs.
- The "**Finite State Automaton**", on the other hand, defines an automaton that can have abstract inputs and possibly also probabilistic or nondeterministic behavior, which will be discussed in later lectures.

Description of FSM

is similar to **directed graphs**,
aka (also known as) **digraphs**

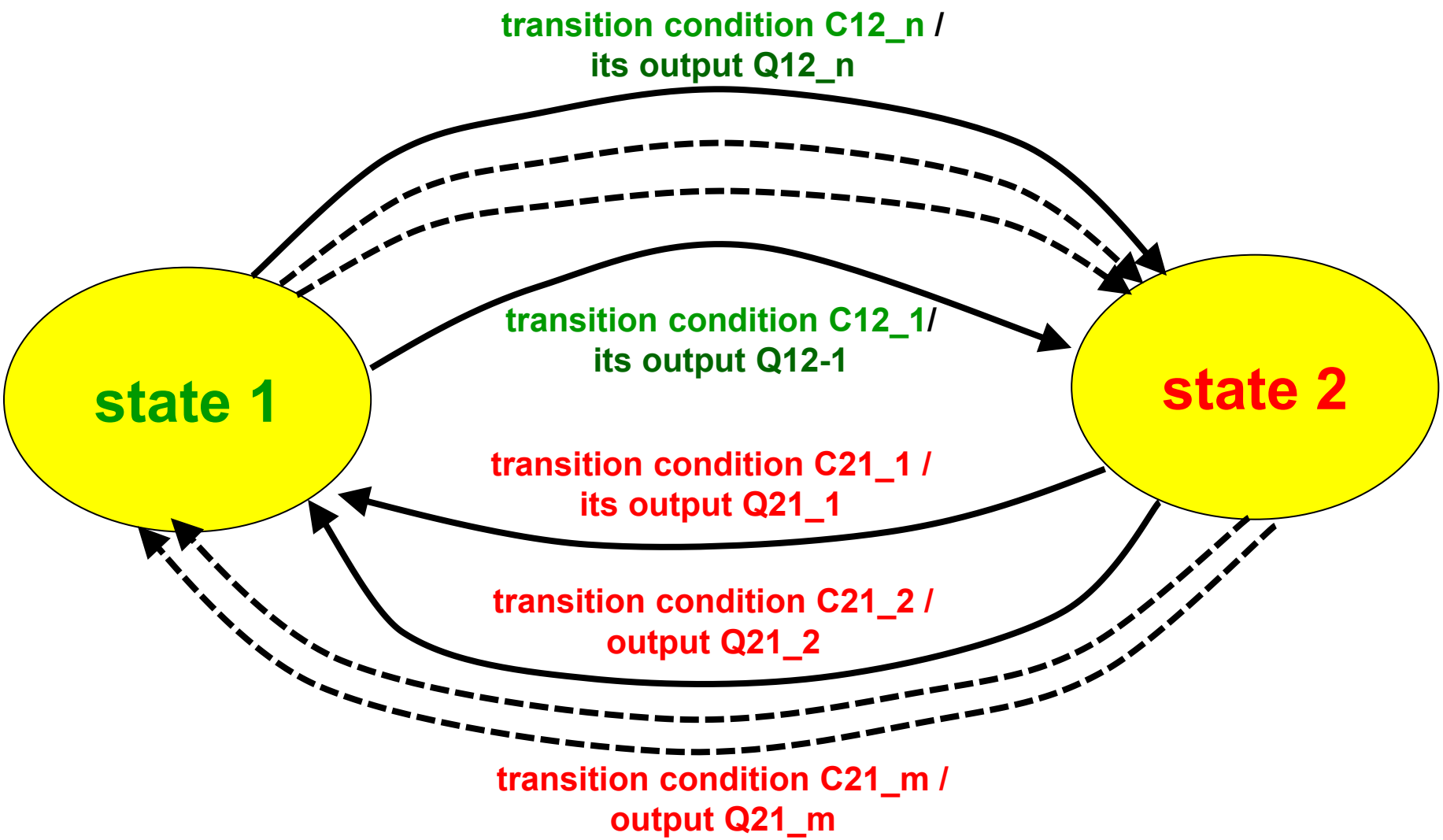
Moore Machine

- Its outputs depend on states only



Mealy Machine

- Its outputs depend on states and current inputs



Example Start-Stop

The circuit from the LSP Morse task

Image credit: Schneider Electric

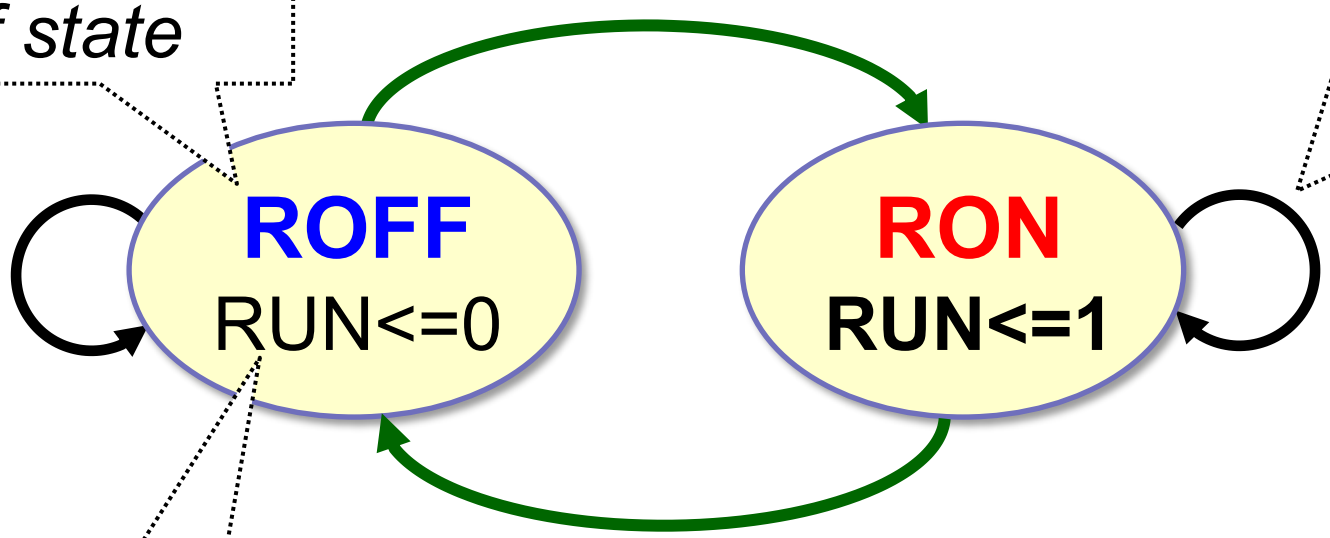
Start-Stop circuit from LSP Morse

Oriented chart of its transitions

*logical condition for transition from ROFF to **RON***

Our name of state

Stop='0' and Start='1'



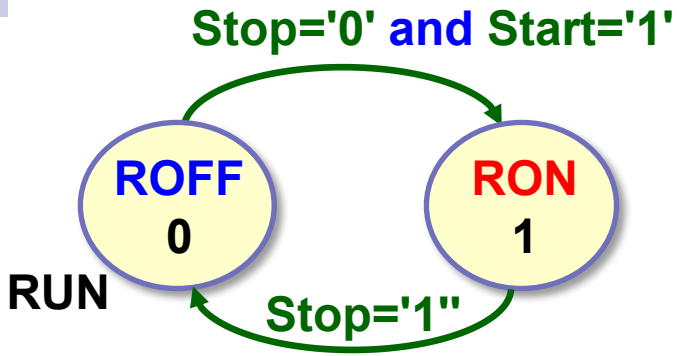
Hint, that FSM remains in the state otherwise; it is often omitted

Stop='1'

generated output

*condition of transition from **RON** to ROFF*

Start-Stop circuit from LSP Morse



Simplified graph of transitions
Transition graph

as **Transition table**

	ROFF	RON
ROFF	-	Stop=0 and Start=1
RON	Stop=1	-

Current Status

future state

Run

0
1

Table Outputs

Stop
Start

ROFF	ROFF	RON	ROFF	ROFF
RON	ROFF	RON	ROFF	ROFF

mere highlighting the steady state

Run

0
1

State transition table

State Transition Table

The state transition table resembles an adjacency matrix, which is used to describe directed graphs. Its square shape remains well-arranged even for a large number of states (vertices). If **p** is the number of all states, then its dimension is **p x p**

rows - current states, from **s1** to **sp**
columns - next states, from **s1** to **sp**
intersection - conditions for x transition from state **sk** to **sj**

M =

	s1		sj		sp
s1					
sk			x		
sp					

<u>Terminological note:</u>
Do not confuse a
state transition table *(finite state machines)*
with the term
state-transition matrix *in control theory, which describes a similar phenomenon but in a continuous area.*

Transition table

2 parts - next state (δ) + outputs (ω)

columns - all possible inputs $\mathbf{x_i}$
rows - all possible internal states $\mathbf{S_j}$ **+ output** $\mathbf{z_m}$
intersection of row and column - subsequent state $\mathbf{s_k}$
output table - output generated in each state

δ ω

S\X	x_1	x_i	x_n
s_1			
s_j		$s_k = \delta(x_i s_j)$	
s_p			

Z
$z_j = \omega(s_j)$

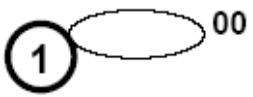
stable state $s_j = \delta(x_i, s_j)$ - highlighted with a circle
unstable state $s_k = \delta(x_i, s_j)$ $s_k \neq s_j$

4 basic transitions

	x2 x1				z
0	0	1	-	2	0
1	1	2	4	-	0
2	-	2	3	4	1
3	-	-	3	-	0
4	1	4	2	0	1

1. Steady state

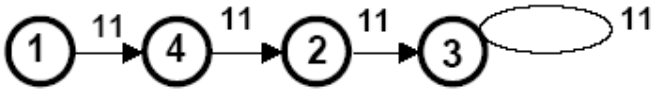
d(xi, sj) = sj



2. Simple transition

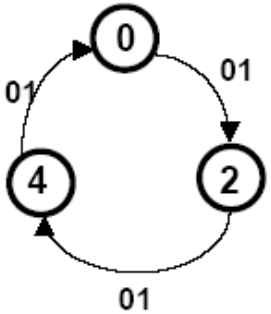
3. Cycle

- through unstable interstates,



4. Oscillations

Note: counters work in oscillation, with each edge of the clock they go to the next state.



Moore vs Mealy

comparission

The ordered tuple

$$\mathbf{M = < X, S, Z, \omega, \delta, s_0 >}$$

- **X** - *finite set of all* input vectors
- **Z** - *finite set of all* output vectors
- **S** - *finite set of all* internal states
- δ - transient function is mapping δ: **X** x **S -> S**
- ω - output function is mapping ω:
  ω: **S -> Z** (Moore)
  ω: **X** x **S -> Z** (Mealy)
- $\mathbf{S_0}$ - initial state $S_0 \in S$

The definition is among the questions asked in the written exam. Students frequently omit red emphasized words, but they are important!

Konečný automat je uspořádaná šestice

$$\mathbf{M = < X, S, Z, \omega, \delta, s_0 \in S >}$$

- **X** - *konečná množina všech* vstupních vektorů
- **Z** - *konečná množina všech* výstupních vektorů
- **S** - *konečná množina všech* vnitřních stavů
  - δ - přechodová funkce - zobrazení δ: **X** x **S -> S**
  - ω - výstupní funkce - zobrazení ω:

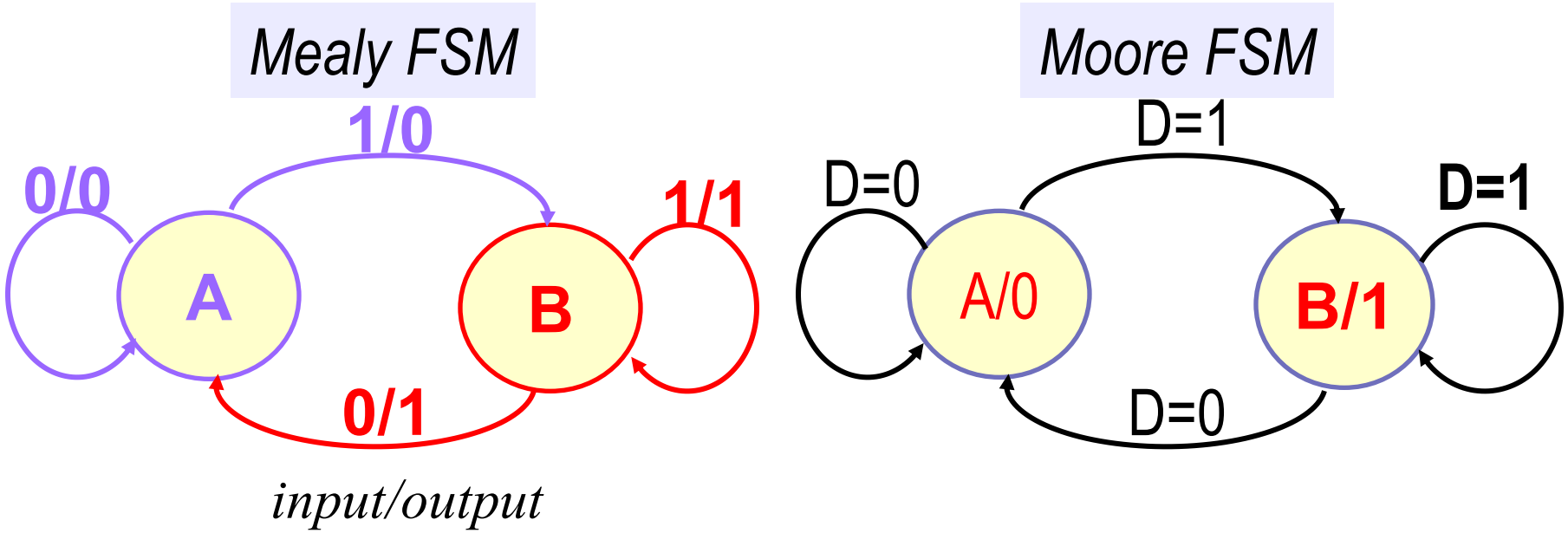
    ω: **S -> Z** Moore

    ω: **X** x **S -> Z** Mealy
- $\mathbf{s_0}$ - počáteční stav $S_0 \in S$

Definice bude u písemek!

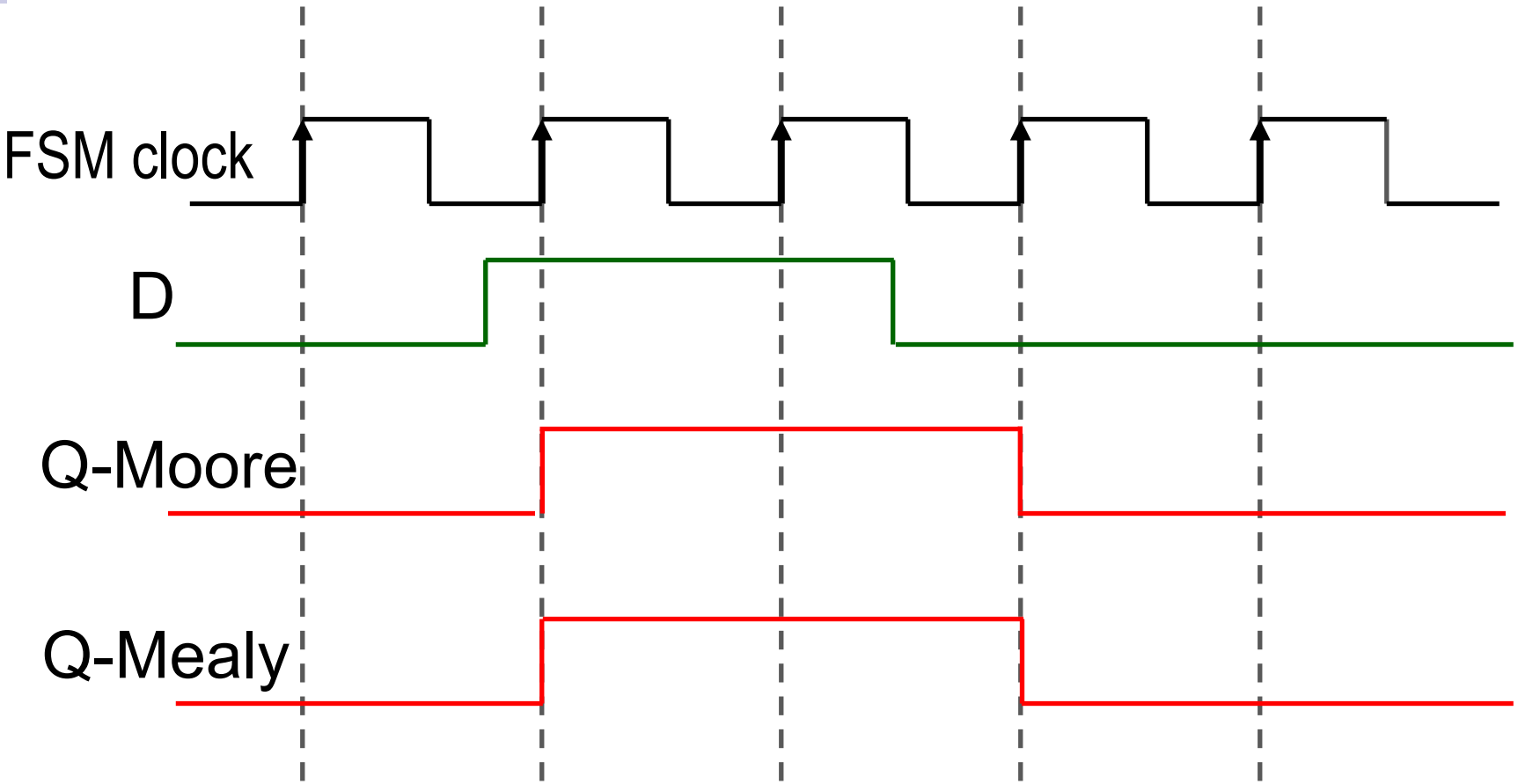
D-register - Mealy and Moore automaton

- Mealy: Output is a function of input and state.
- Moore: The output is only determined by the state.



Both FSMs behave identically because Mealy's automaton generates the same outputs on all transitions from each given state.

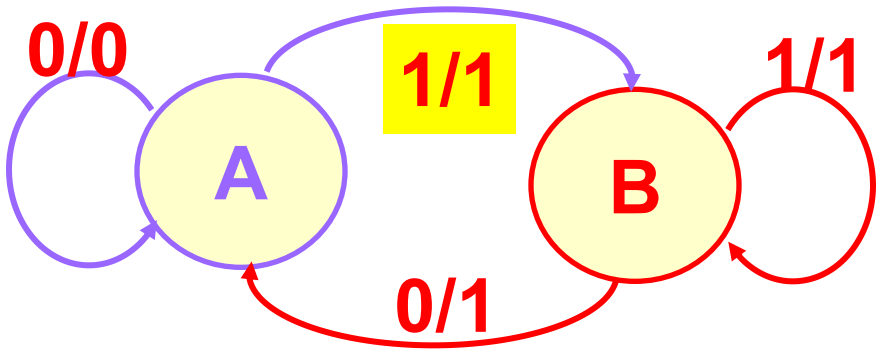
Moore & Mealy FSMs : D-register



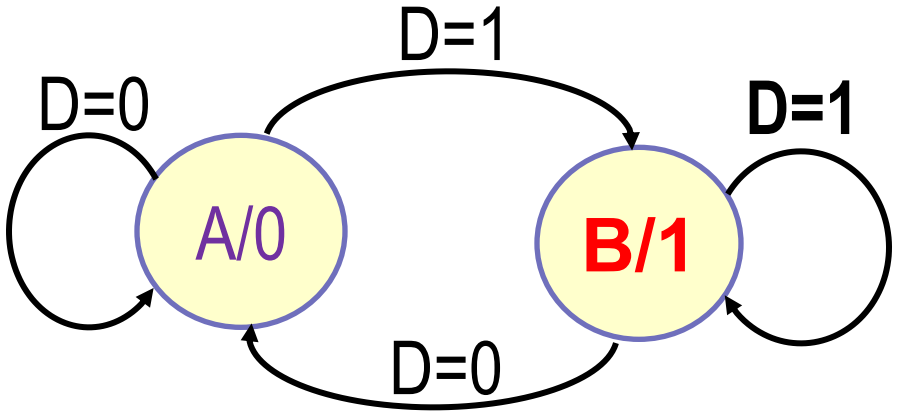
D-register - Mealy and Moore automaton

Mealy's machine can also predict

Moore's automaton is always without prediction

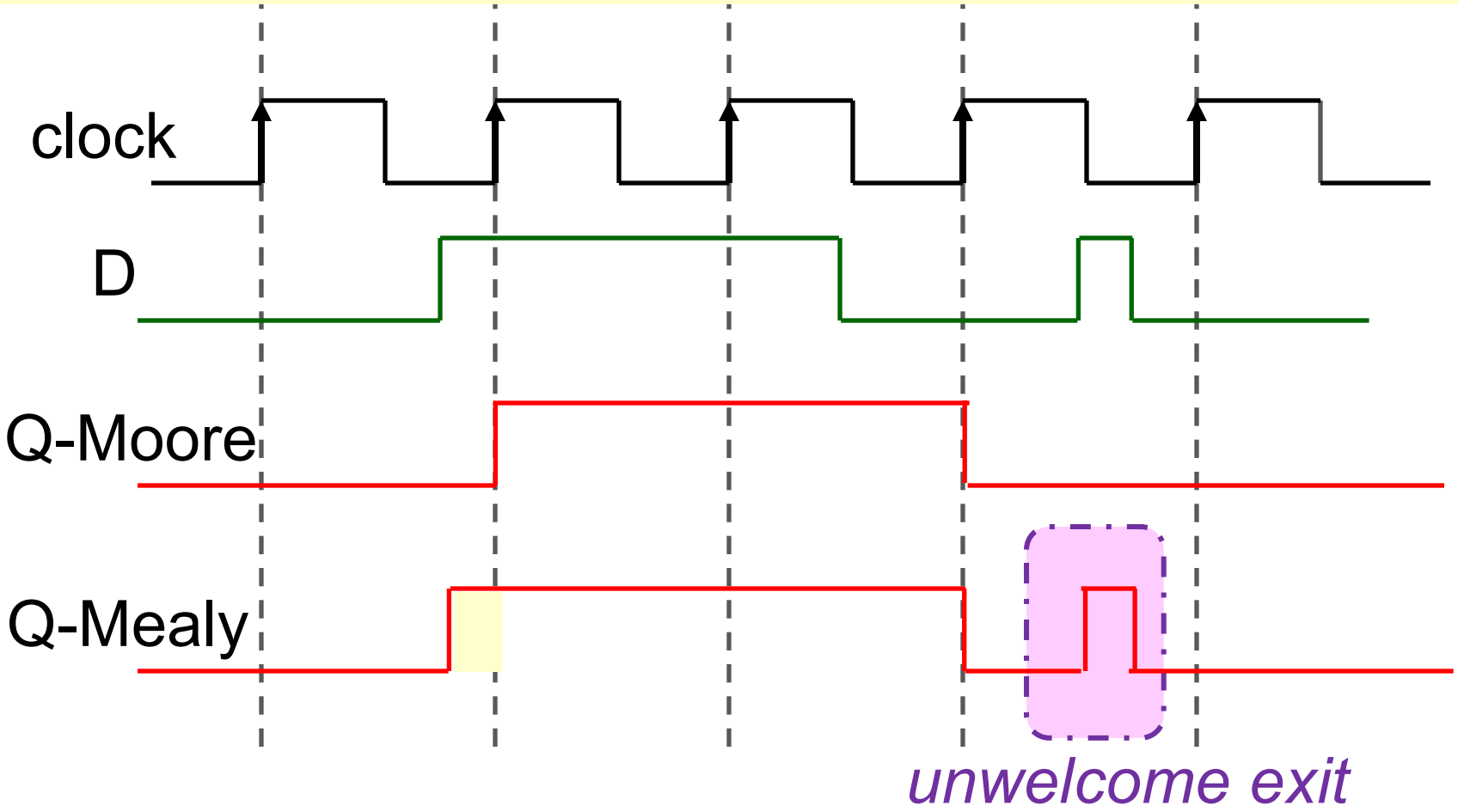


input/output



D-register Moore's automaton & Mealy's with prediction

The output of the Mealy's FSM should always be synchronized by the FSM clock in circuits !



Moore and Mealy's machine

- Both machines differ only in the generation of outputs.

Moore's machine:

- Its outputs do not depend on the inputs, so they are effectively generated from the state in which the FSM is.

Moore's FSMs are preferred in circuits. Input disturbances do not propagate through the ω *function to outputs.*

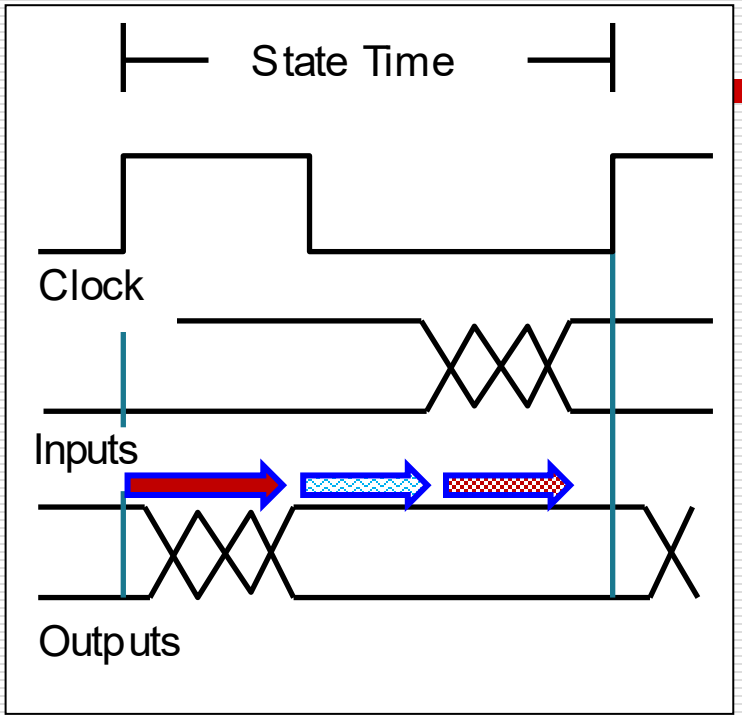
Mealy's machine:

- Its outputs are generated from the current state and the immediate input, i.e., the outputs can change even if the FSM does not change its state.

Mealy's FSMs can be simpler than their Moore's equivalents; therefore, they are more commonly used in programs where variable values are free of disturbances. In circuits, their outputs must be synchronized by the FSM clock to prevent glitches, i.e., unwanted pulses.

State Machine with Controlled System

Figure drawn for Positive Edge Triggered Synchronous FSM's register

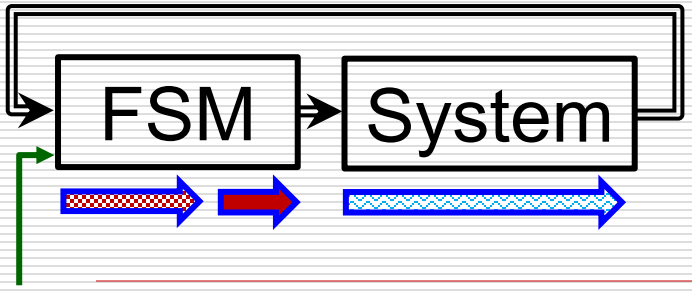


On the rising edge, FSM's register loads a new state. The FSM outputs begin changing.

After the propagation delay of FSM's ***ω function***, the FSM's outputs become stable. *They are stable immediately if FSM has no ω function*.

The change of FSM's outputs influences the system controlled by the FSM. After its propagation time, the FSM's inputs begin to change.

The δ function result becomes stable after the propagation delay given by the loop
[*ω* -> controlled system -> inputs -> δ]



Now, the next state (the result of the δ function) can be loaded into the state register by the FSM's clock. The loop above determines the **minimum period** of FSM's clock.

Moore / Mealy Theorem

Theorem:

- For any Moore's FSM, there exists an equivalent Mealy's FSM with the same or **fewer** states.
- Conversely, for every Mealy's FSM that has N states and recognizes X possible combinations of inputs, there is an equivalent Moore's FSM having N*X states.

The proof can be done by construction.

The coding of FSM states and their description in VHDL

NP-complete state coding problem

- The encoding of the state can significantly affect the optimal implementation of the automaton.
  - Finding the optimal code means trying all possible variations. We can check them only for small FSMs.
  - In general, pseudo-optimal state codes are chosen, i.e., assigned by some heuristic in the hope that they will be close to the optimal ones...

Example of encoding states in VHDL

The **recommended encoding** is an enumerated type, for example

```
type state_t is ( sinit, s1, s2, s3);
signal state   : state_t := sinit;
```

In VHDL, enumerated types are reserved for states.
Quartus compiles them as FSMs, using their encoding into code close to 1 of N "one-hot." [Quartus reference manual]

States	Binary Codes
sinit	0 0 0 0
s1	0 0 1 1
s2	0 1 0 1
s3	1 0 0 1

When describing circuits in VHDL, avoid trying to imitate the coding styles used in C.

Enum type

Quartus translates all 'enum' types as FSMs, so use 'enum' types only in them.

A note for very advanced study:

The enum encoding is determined by "Setting (Ctrl+Shift+E) -> Analyse & Synthetis Settings -> [More Settings] -> State Machine Processing", which is Auto by default. It defines the default value of the sys_encoding *attribute. Do not change!*

- *If we want to use some 'enum' types as just numeric constants, we can add a special attribute* **enum_encoding**
  (Google search: "Quartus attribute enum_encoding*" will give a link)*
- *But the enum_encoding attribute will block this enum type from being recognized as FSM states. So do not use enum_encoding for types that are intended for encoding FSM states.*
- *For details, see Quartus handbook, pages 16-36 to 16-38, in pdf 741 to 743:*
  https://dcenet.felk.cvut.cz/edu/fpga/doc/quartusii_handbook_archive_131.pdf

Unrecommended encodings of states in VHDL

On websites, various status encodings are sometimes seen, which are not only much more laborious but also less flexible, e.g.

```
constant sinit : integer := 0;
constant s1 : integer := 1;
constant s2 : integer := 2;
constant s3 : integer := 3;
signal state : integer range 0 to 3; -- We must specify range!!!
```

Encoding by subtypes is slightly safer, but still unsuitable

```
subtype  state_type is unsigned(1 downto 0);
constant sinit      : state_type := "00" ;
constant s1         : state_type := "01" ;
constant s2         : state_type := "10" ;
constant s2         : state_type := "11" ;
```

Example Start-Stop

from the LSP Morse tasks

Start-Stop as a Circuit

```
library ieee;use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity StartStopFSM is port (
        START, STOP, CLK : in std_logic;
        CLRN : in std_logic; -- FSMs prefer synchronous clear
        RUN : out std_logic);
end entity;
architecture rtl1 of StartStopFSM is
begin -- architecture
ilog: process(CLK)
        begin
            -- the next slide
        end process;
end architecture;
```

Start-Stop - architecture

```
architecture rtl1 of StartStopFSM is
begin -- architecture
iproc: process(CLK)
   begin
      iRedge : if rising_edge(CLK) then
            if STOP='1' or CLRN='0' then RUN <='0';
            elsif START ='1' then RUN <='1';
            end if;
      end if iRedge;
  end process;
end architecture;
```

Start-Stop as FSM

```
architecture rtlFSM of StartStopFSM is
begin
ifsm: process(CLK)
    type state_t is (ROFF, RON);
    variable state: state_t:=ROFF;
    begin
      iRedge : if rising_edge(CLK) then
```

δ

```
        end if iRedge;
```

ω

```
    end process;
end architecture;
```

Circuit vs FSM

```
iRedge : if rising_edge(CLK) then
          if STOP='1' or CLRN='0' then RUN <='0';
         elsif START ='1' then RUN <='1';
         end if;
 end if iRedge;
```

```
iRedge : if rising_edge(CLK) then
    iClrn: if CLRN='0'  then state:=ROFF; else
            case state is
              when ROFF => if START='1' and STOP='0'
                           then  state:=RON; end if;
               when RON => if STOP='1'
                           then state:=ROFF; end if;
          end case; end if iClrn;
 end if iRedge;
 if state=RON then RUN<='1'; else RUN<='0'; end if;
```

δ

ω

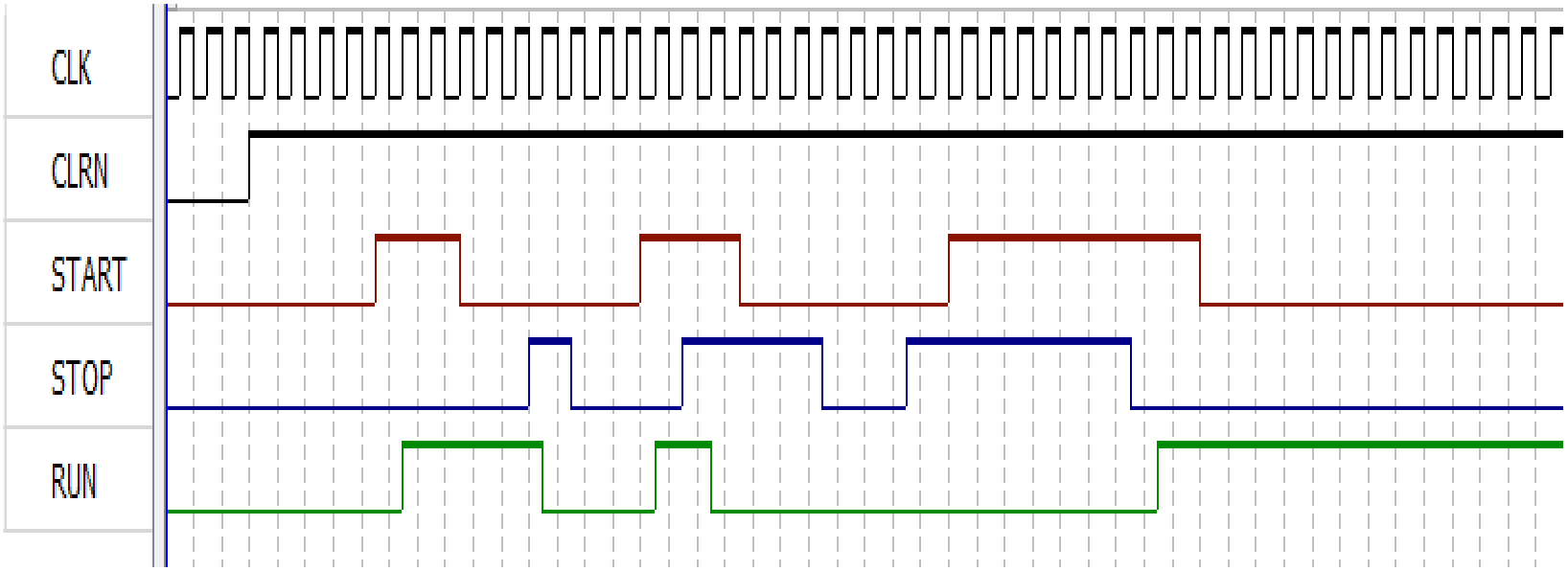
Whole Architecture

```
architecture rtlFSM of StartStopFSM is
begin
ifsm: process(CLK)
  type state_t is (ROFF, RON); -- enumerated types are reserved only for FSMs
  variable state: state_t:=ROFF;
  begin
     iRedge : if rising_edge(CLK) then
        iClrn: if CLRN='0'  then state:=ROFF; -- always clear
                 else
                  case state is
                      when ROFF => if START='1' and STOP='0'
                                          then  state:=RON; end if;
                    when RON => if STOP='1'
                                        then state:=ROFF; end if;
          end case; end if iClrn;
    end if iRedge;
    if state=RON then RUN<='1'; else RUN<='0'; end if;
  end process;
end architecture;
```

δ

ω

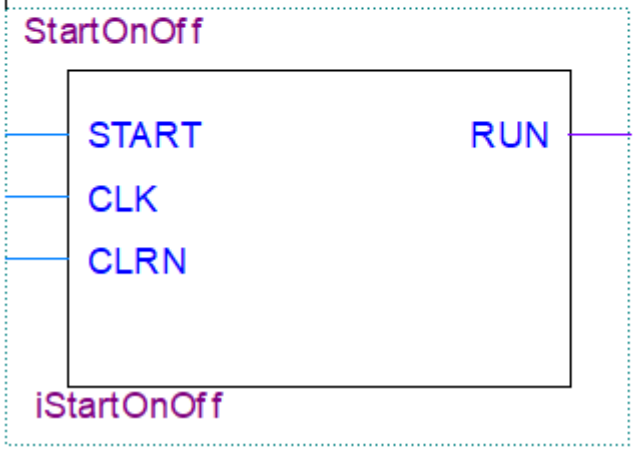
Simulation



Exended Start-Stop

The Start button turns on after 2 seconds and turns off after 0.5 seconds.

Parameter	Value	Type
FCLK	50000000	Signed Integer
ON_MS	2000	Signed Integer
OFF_MS	500	Signed Integer

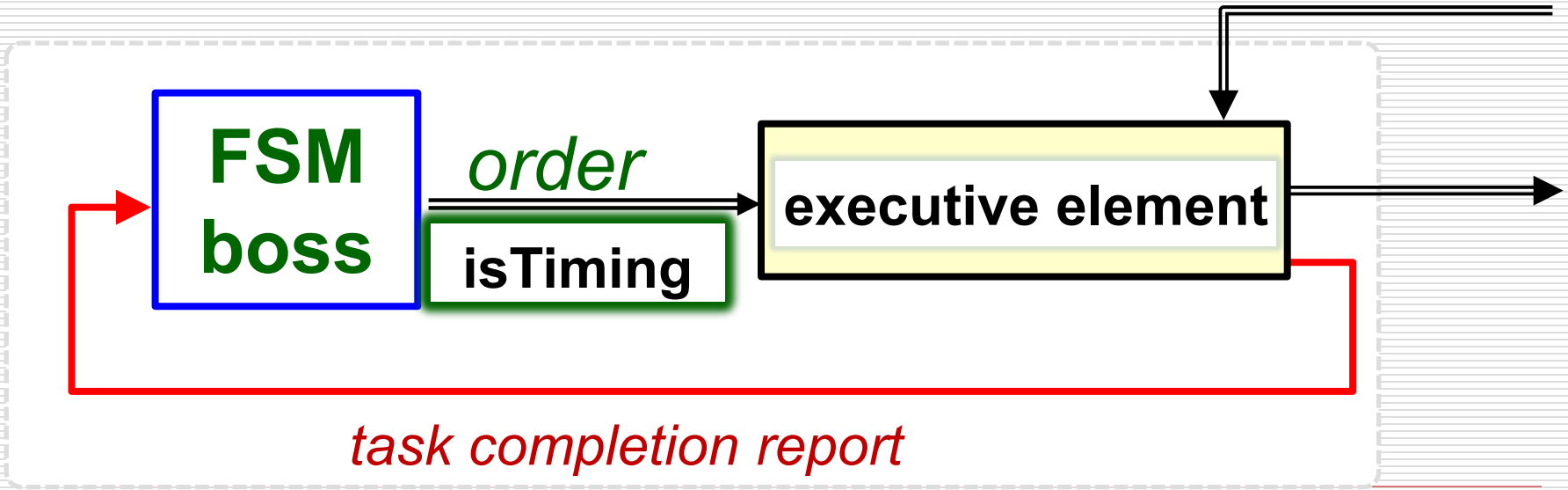


FSM IS BOSS

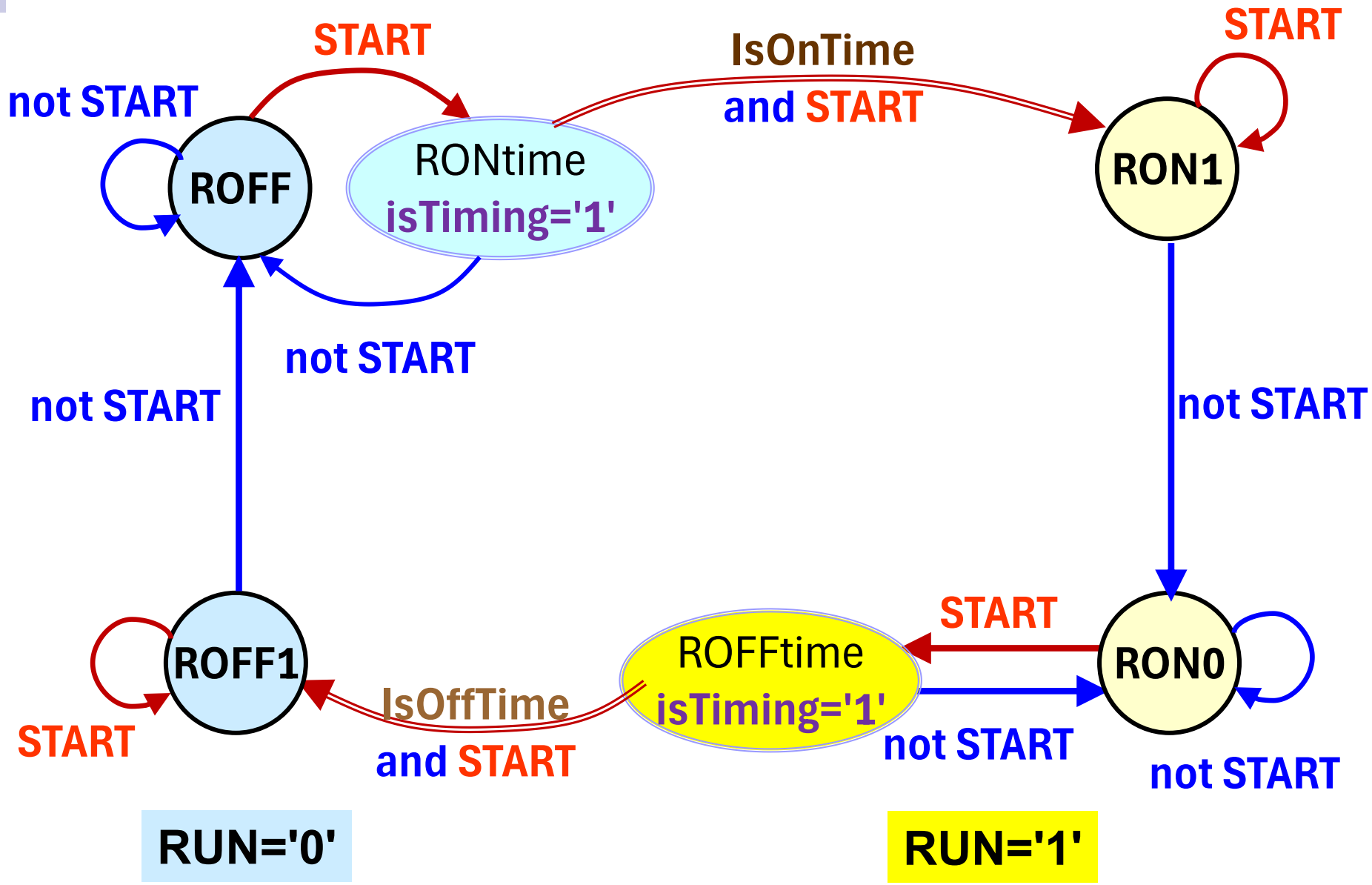
it only organizes the others

- issues an order and
- waits for it to be carried out.

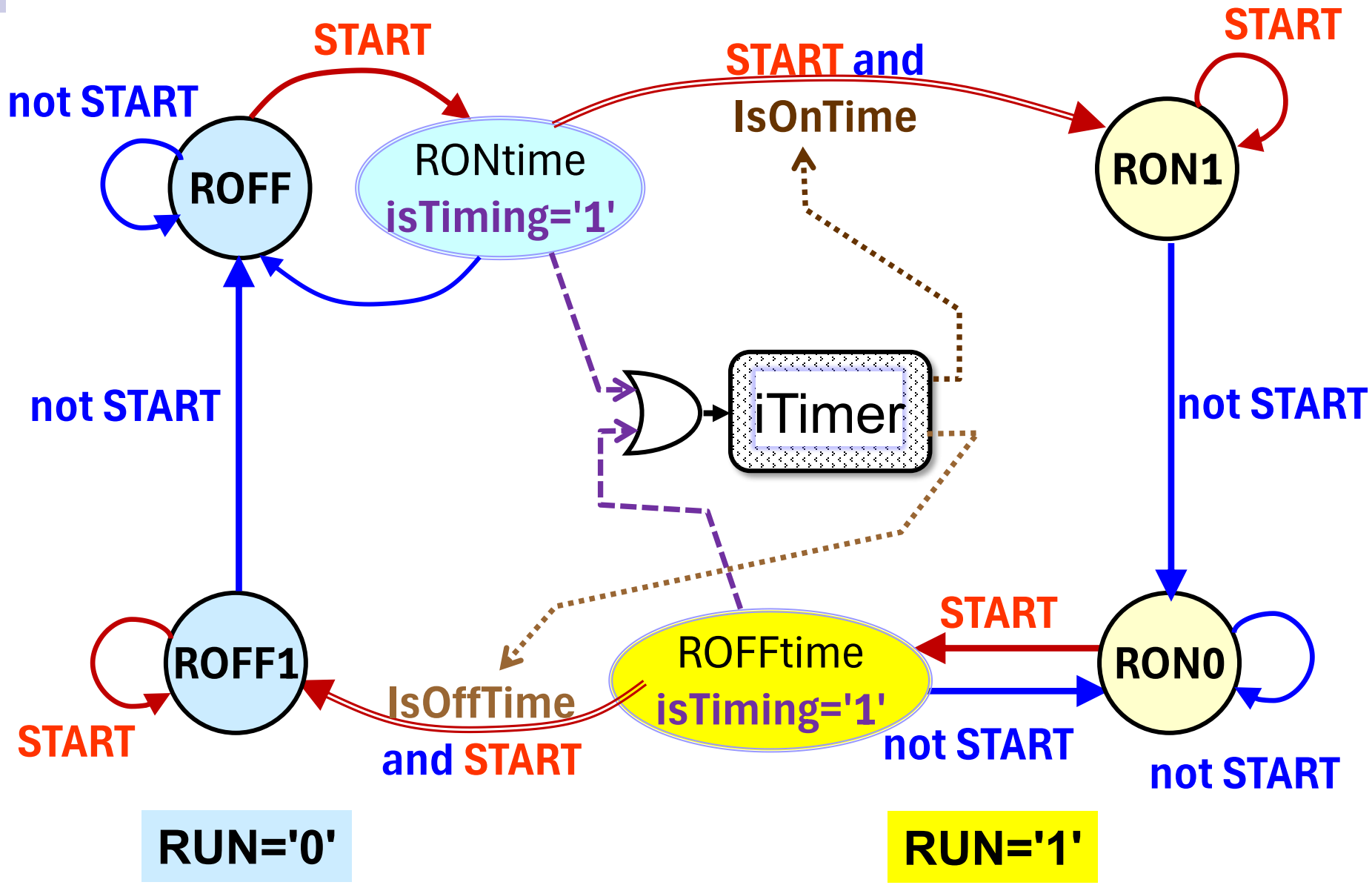
Possible
direct
inputs
outputs



Transition Graph



Transition Graph with Timer



Entity + process header

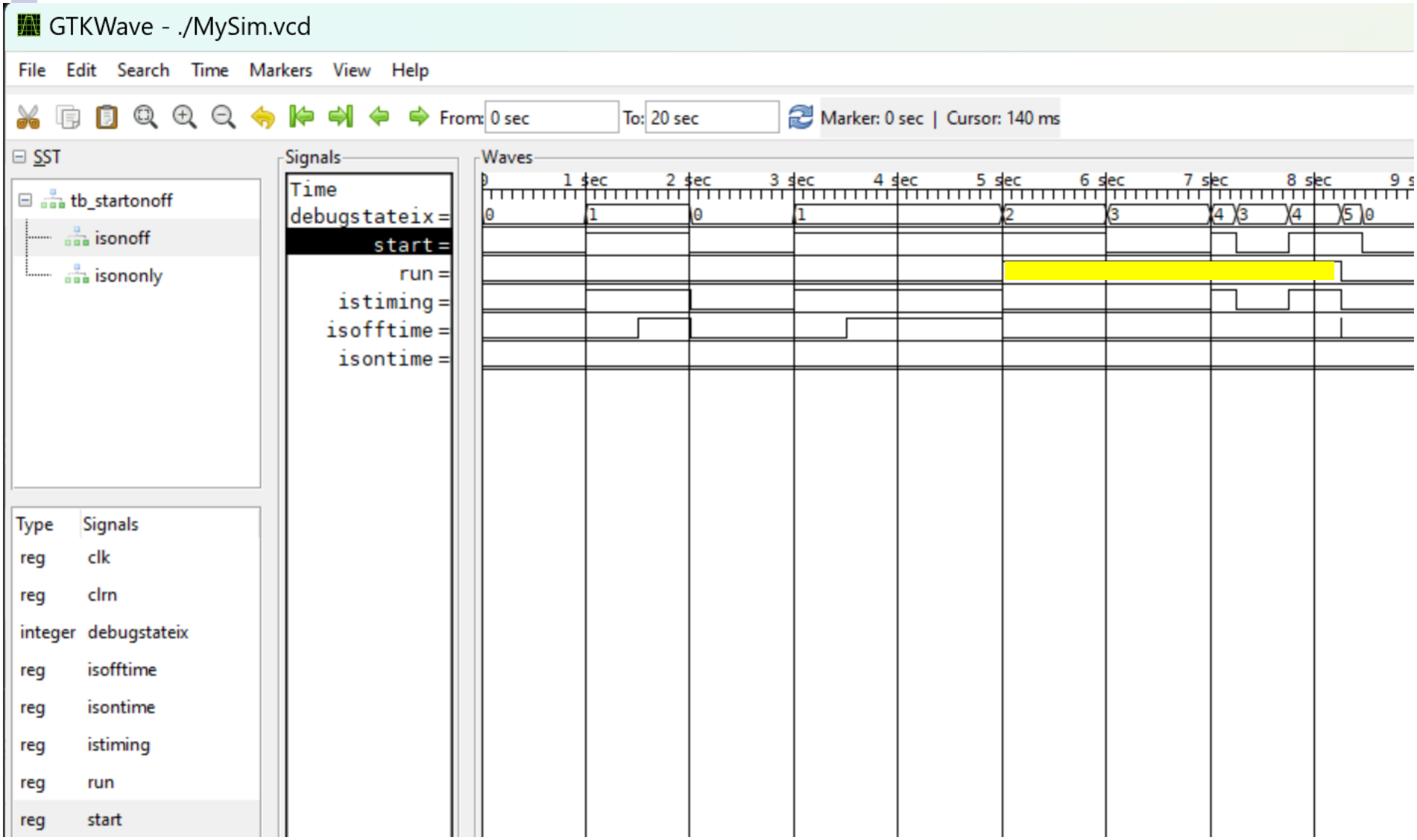
```
library ieee;use ieee.std_logic_1164.all; use ieee.numeric_std.all;
entity StartOnOff is
generic(FCLK: positive:=50000000;
            ON_MS:positive:=2000;  OFF_MS:positive:=500);
port (START, CLK, CLRN : in std_logic;
        RUN : out std_logic);
end entity;

architecture rtlOnOff of StartOnOff is
  signal isTiming, isOnTime, isOffTime:boolean:=false;
  signal debugStateIx:integer:=0;
 begin -- architecture

ifsm: process(CLK)
  type state_t is (ROFF, RONTIME, RON1, RON, ROFFTIME, ROFF1);
  variable state: state_t:=ROFF;  variable vrun:std_logic:='0';
  begin -- process
```

```
ifsm: process(CLK)
 type state_t is (ROFF, RONTIME, RON1, RON, ROFFTIME, ROFF1);
 variable state: state_t:=ROFF; variable vrun:std_logic:='0';
 begin
 iRedge:if rising_edge(CLK) then  isTiming<=false; vrun:='0';
  iCLRN: if CLRN='0'   then state:=ROFF; else
              case state is
                when ROFF =>  if START='1' then state:=RONTIME; end if;
                when RONTIME=> isTiming<=true;
                   if START='0' then state:=ROFF;  elsif isOnTime then state:=RON1; end if;
                when RON1 => vrun:='1';  if START='0' then state:=RON; end if;
                when RON => vrun:='1'; if START='1' then state:=ROFFTIME; end if;
                when ROFFTIME=> vrun:='1'; isTiming<=true;
                     if START='0' then state:=RON; elsif isOffTime then state:=ROFF1; end if;
                when ROFF1 =>  if START='0' then state:=ROFF; end if;
            end case;
        end if iCLRN; --if CLRN='0'  then
    end if iRedge; --  if rising_edge(CLK) then
    RUN<=vrun;
end process;
```

```
iTimer:process(CLK)
 function max(x,y:integer) return integer is
      begin if x>=y then return x; else return y; end if;
      end function;
 constant MAXCOUNT:integer:=max( ON_MS, OFF_MS);
 constant MSCOUNT : integer := (FCLK+500)/1000;
 variable cntrMS: integer range 0 to MSCOUNT:=0;
 variable cntr: integer range 0 to MAXCOUNT:=0;
  begin --process

  end process;
```

iTimer

```
  begin --process
iRedge:if rising_edge(CLK) then
   iTiming:if not isTiming then cntr:=0; cntrMS:=0; else
                if MSCOUNT>1 and cntrMS<MSCOUNT-1
                then cntrMS:=cntrMS+1;
                else cntrMS:=0;
                      if cntr<MAXCOUNT then cntr:=cntr+1;
                      end if;
               end if;
      end if iTiming;
    end if iRedge;
    isOnTime<=(cntr>=ON_MS); isOffTime<=(cntr>=OFF_MS);
  end process;
```


GTKWave - ./MySim.vcd
File Edit Search Time Markers View Help
From: 0 sec
To: 20 sec
Marker: 0 sec | Cursor: 140 ms
SST
tb_startonoff
isonoff
isononly
Type Signals
reg clk
reg clrn
integer debugstateix
reg isofftime
reg isontime
reg istiming
reg run
reg start
Signals
Time
debugstateix =
start =
run =
istiming =
isofftime =
isontime =
Waves
1 sec
2 sec
3 sec
4 sec
5 sec
6 sec
7 sec
8 sec

FSM

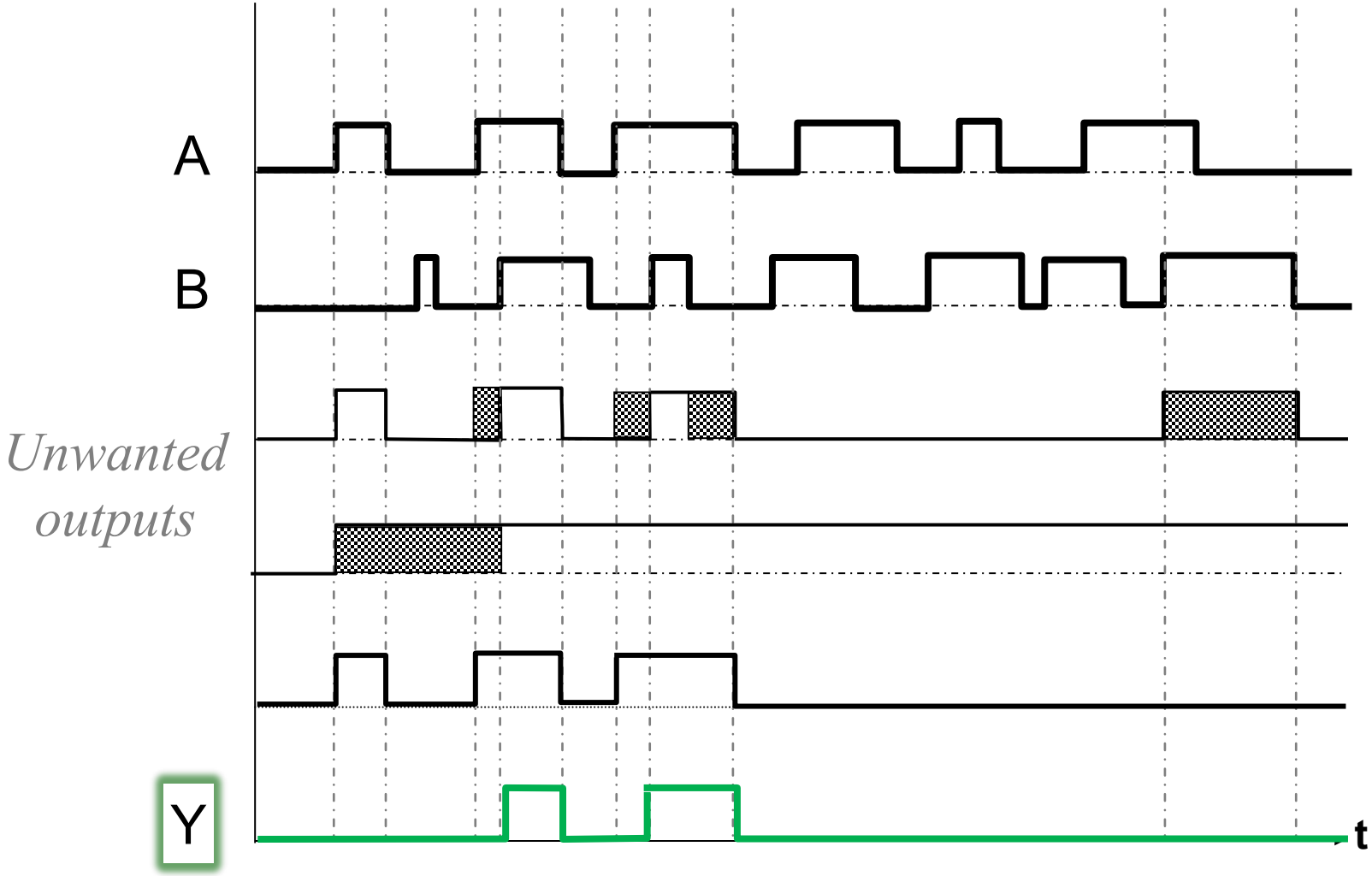
- Using the concept of an FSM makes the design clearer. States and transitions can be analyzed and modified more easily.
- The finite state machine is transcribed into the selected HDL language almost mechanically, so in the next part of the lecture, we will focus on the design of states and transitions of finite state machines. The resulting code may sometimes be more extended, but with a lower probability of errors.
- In addition, compiler converts automata very efficiently using constructions optimized at the level of logical elements.
- The automaton concept also allows circuit components to be shared, as the previous timer.
- A special type of automaton called a control unit, discussed in the following lecture, is a fundamental element of all processors.

Example

If input A went to '1' before B,
then output Y is '1',
otherwise it remains at '0'.

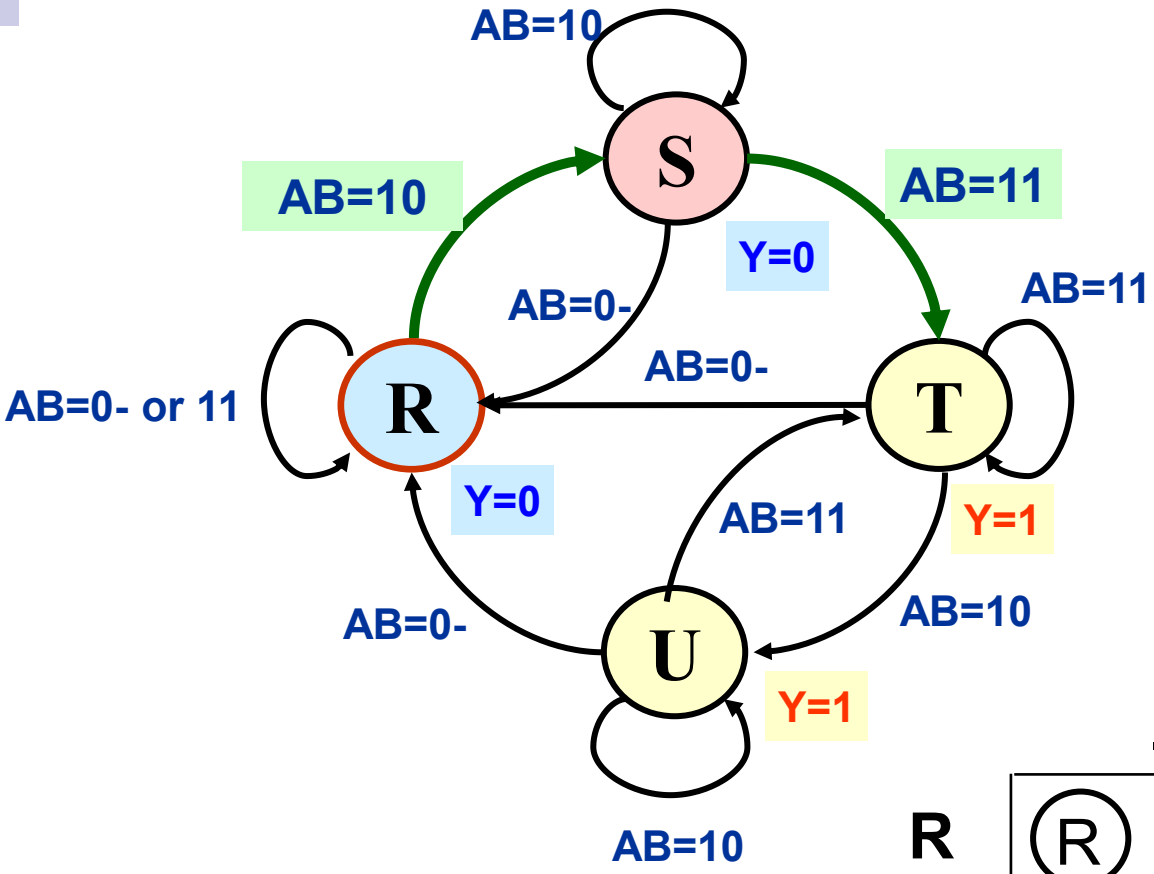
Question: Are the requirements unambiguous, or not?

There are more possibilities



The customer requested output Y. Who didn't ask...

Transition graph + Transition table



δ

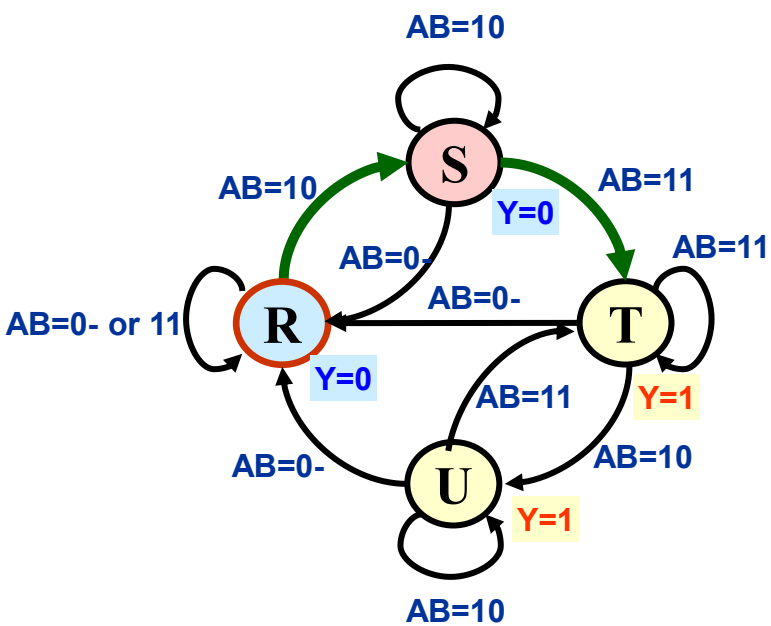
	BA			
R	R	S	R	R
S	R	S	T	R
T	R	U	T	R
U	R	U	T	R

ω

Y
0
0
1
1

How do we construct a transition graph?

- The construction is a particular analogy to a board game plan, in which a single piece specifies the current state's position. When the conditions are met, it moves to the next square.
- FSMs usually have synchronization states in which they wait for the start of their work cycle. Here, this is state R, which we fall into when A=0. First, we construct the path from it to the goal, which is green in the diagram. Then we add the other options.
- The transition table will also help us with this. It has a row label on the left, i.e., the current state, and inside it are the future states that will be reached when the conditions listed above are met.
- The transition graph also shows waiting loops in the state for educational purposes only. Still, in practice, they are omitted.
- The existence of a waiting loop is only graphically highlighted with circles in the transition table. We circle all states identical to their row, because they are stable for the column combinations of input values.



δ	B A: ___	___	___	___	Y ω
R	R	S	R	R	0
S	R	S	T	R	0
T	R	U	T	R	1
U	R	U	T	R	1

Minimizing states

δ

R	R	S	R	R
S	R	S	T	R
T	R	U	T	R
U	R	U	T	R

ω

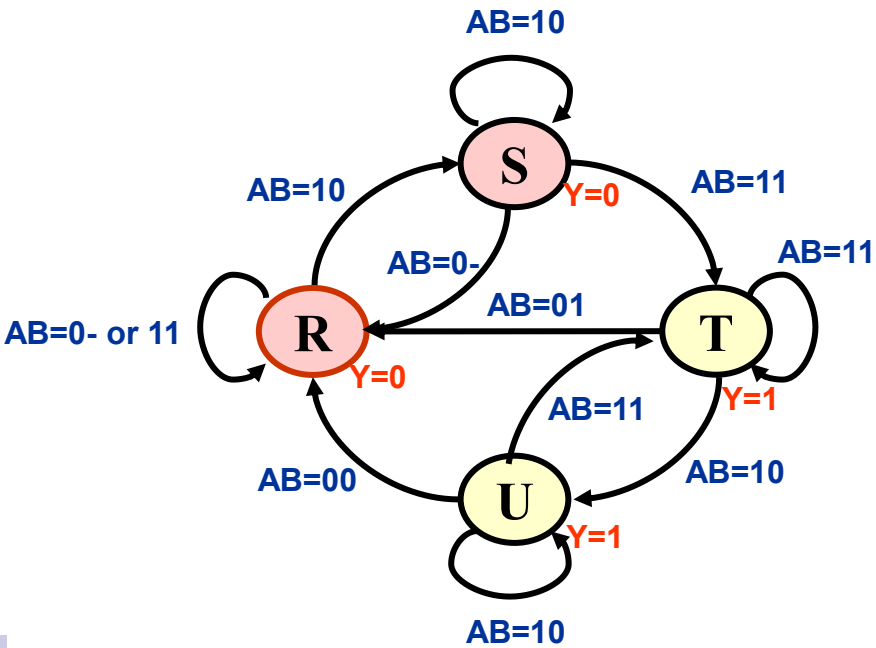
B A
0
0
1
1

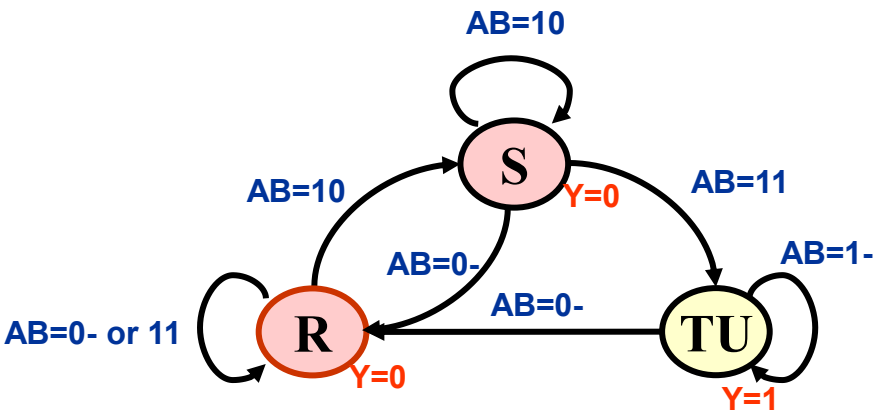
δ

R	R	S	R	R
S	R	S	TU	R
TU	R	TU	TU	R

ω

B A	Y
	0
	0
	1





Main Part VHDL AbeforeB

```
iFSM: process(CLK, ACLRN)
type state_t is (stR, stS, stTU);-- enumerated type for FSM
variable state   : state_t; -- The current state
begin
 iRedge: if (rising_edge(CLK)) then
  if RESET = '1' or A = '0' then state := stR;
  else case state is -- The next state function
          when stR=>    if B='0' then state :=stS; end if;
          when stS=>    if B='1' then state :=stTU; end if;
          when stTU=>  null;  end case;
          end if;
 end if iRedge;
 if state=stTU then Y<='1';else Y<='0';
 debugIxState<=state_t'Pos(state); end process;
```

Whole VHDL AbeforeB

```
library ieee; use ieee.std_logic_1164.all;
entity AbeforeB is port(A, B, CLK, RESET: in std_logic;  Y: out std_logic);
end entity;
architecture rtl of AbeforeB is
signal debugIxState:integer:=0;
begin
iFSM: process(CLK)
type state_t is (stR, stS, stTU);-- enumerated type for FSM
variable state   : state_t; -- The current state
begin
 if (rising_edge(CLK)) then
   if RESET = '1' or A = '0' then state := stR;
   else
      case state is -- The next state function
       when stR=>    if B='0' then state :=stS; end if;
       when stS=>    if B='1' then state :=stTU; end if;
       when stTU=>  null;
      end case;
   end if;
 end if; -- output is a combinational function - no delay
 if state=stTU then Y<='1'; else Y<='0'; end if; debugIxState<=state_t'Pos(state);
end process; end architecture;
```

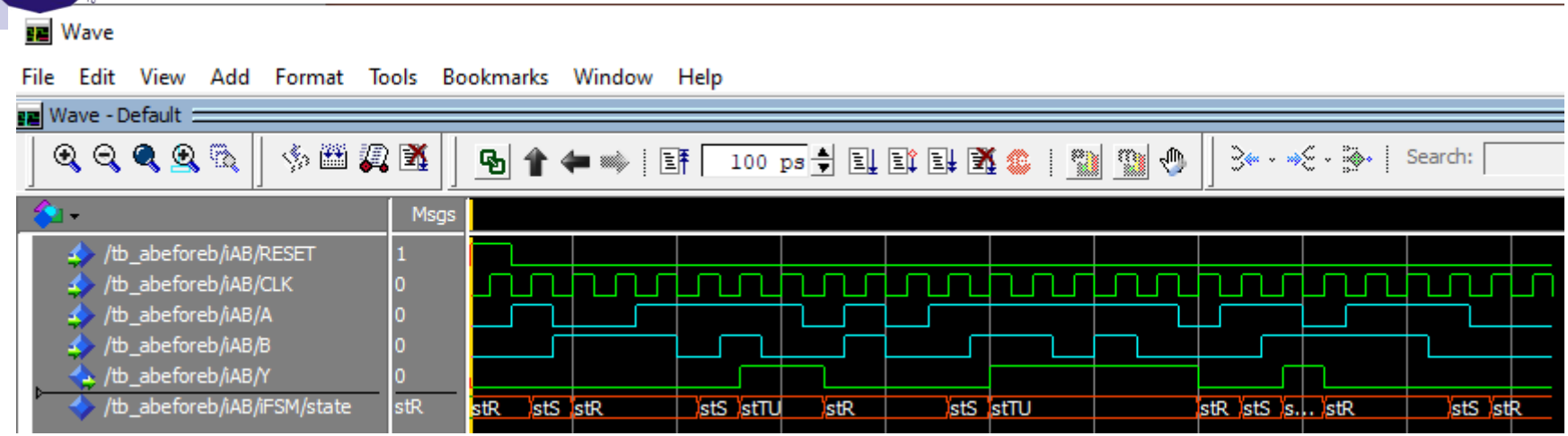
Attributes of States 1/4

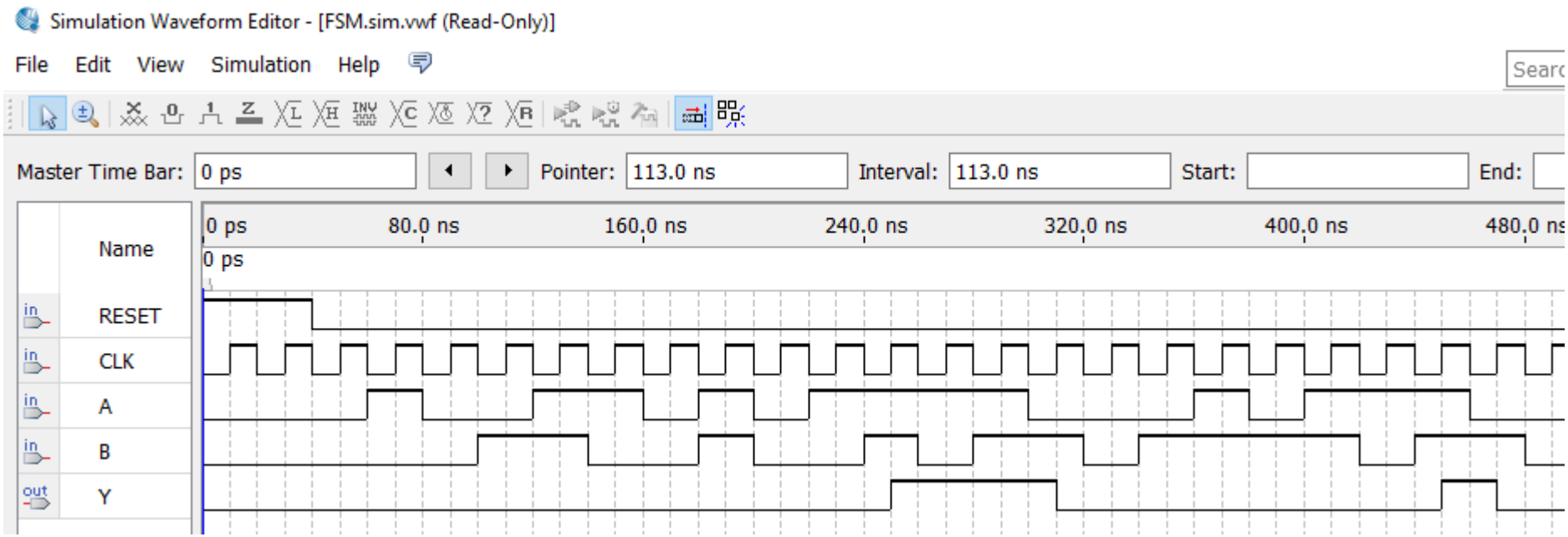
```
process(CLK)
  type state_t is (s0, s1, s2, s3, s4);
  variable state: state_t:=s0;
  function State2uint(st:state_t) return unsigned is
  begin  return to_unsigned(state_t'POS(st),3);
           -- order number of state
  end function;
  variable ix:unsigned(ixR'RANGE);
begin -- process

end process;
```

Attributes of States 2/4

```
library ieee; use ieee.std_logic_1164.all;use ieee.numeric_std.all;
entity AttrOfStateType is  port ( CLK : in std_logic;  ixR,valR,predR,succR,leftR,rightR:out unsigned(2 downto 0));
end;
architecture rtl of AttrOfStateType is
begin
 process(CLK)
    type state_t is (s0, s1, s2, s3, s4);
    variable state: state_t:=s0;
    function State2uint(st:state_t) return unsigned is
    begin  return to_unsigned(state_t'POS(st),3); -- order number of state
    end function;
    variable ix:unsigned(ixR'RANGE);
 begin
      if rising_edge(CLK) then
          ix:=State2uint(state); ixR<=ix; -- the order number
          valR<=State2uint(state_t'VAL(to_integer(ix))); -- state from the order
          predR<=State2uint(state_t'PRED(state)); -- the previous state
          state:=state_t'SUCC(state); succR<=State2uint(state); -- next state
      end if;
leftR<=State2uint(state_t'LEFT);  -- leftmost state in the definition
rightR<=State2uint(state_t'RIGHT); -- rightmost state in the definition
 end process; end architecture;
```

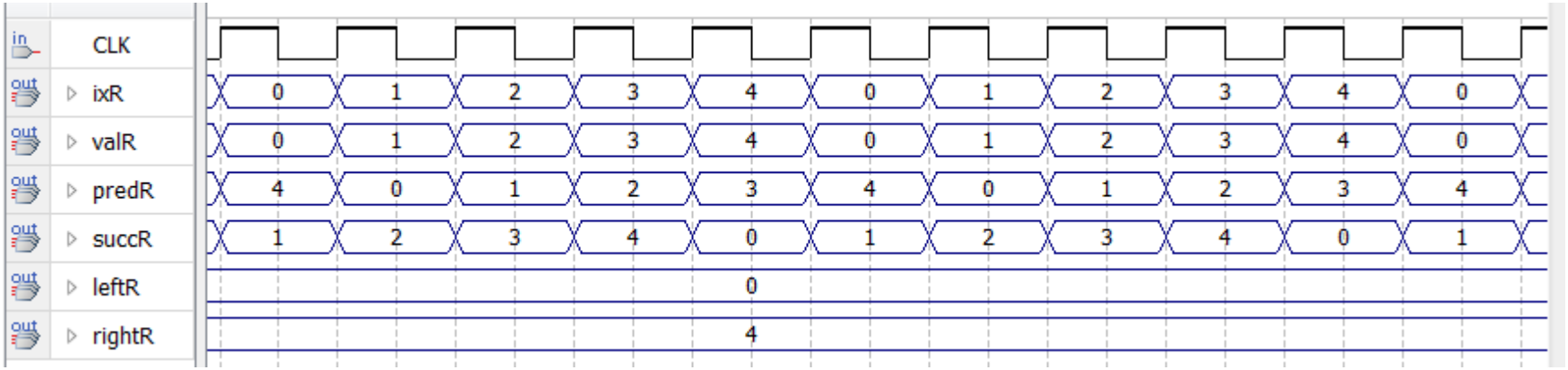
Simulation VWF + ModelSim





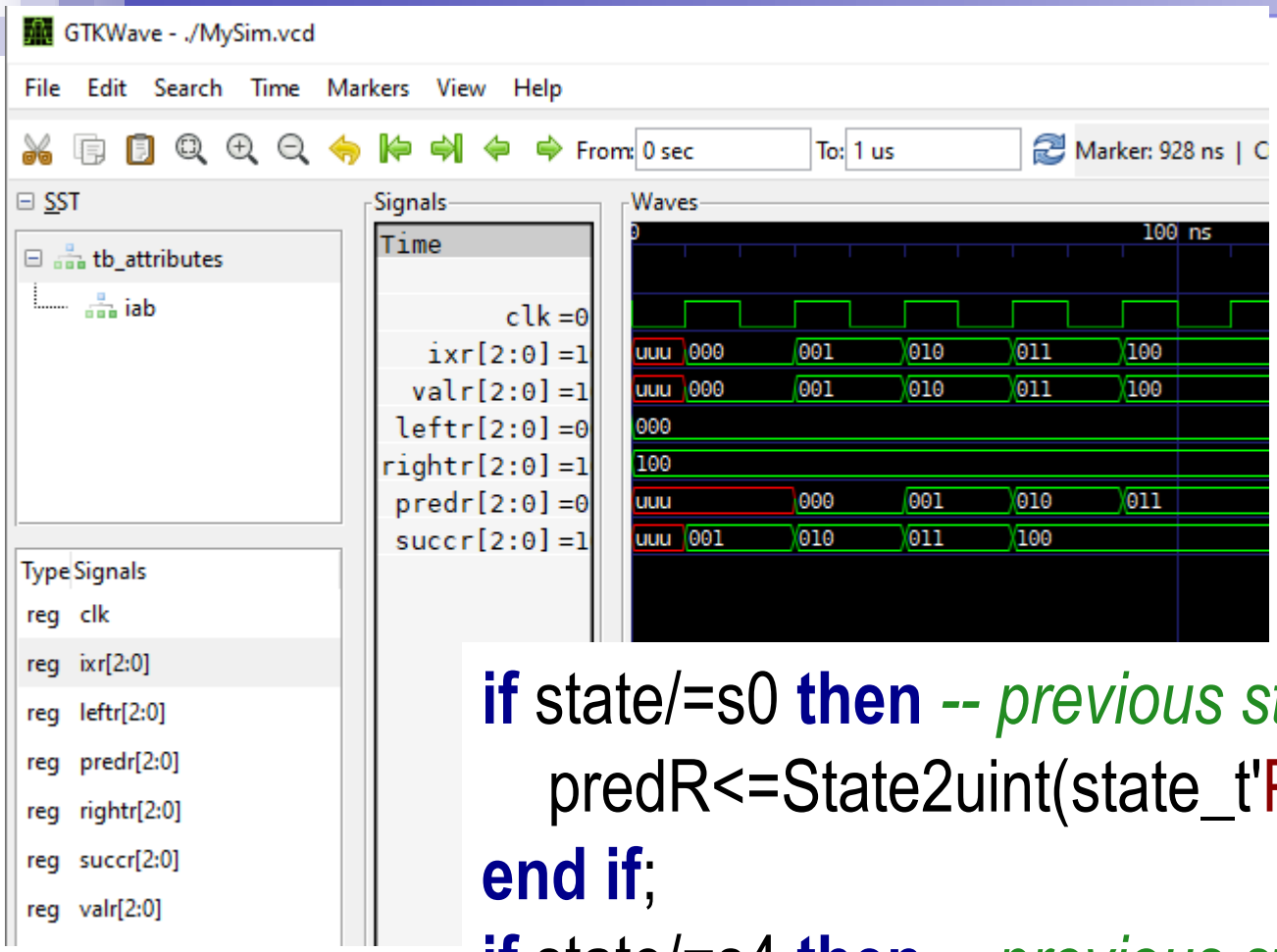
ModelSim:Attributes of States 3/4

type state_t is (*s0, s1, s2, s3, s4*);




```
if rising_edge(CLK) then
   ix:=State2uint(state); ixR<=ix; -- the order number state_t'POS(state)
   valR<=State2uint(state_t'VAL(to_integer(ix))); -- state from the order
   predR<=State2uint(state_t'PRED(state)); -- the previous state
   state:=state_t'SUCC(state); succR<=State2uint(state); -- the next state
end if;
leftR<=State2uint(state_t'LEFT);  -- the leftmost state in the definition
rightR<=State2uint(state_t'RIGHT); -- the rightmost state in the definition
```

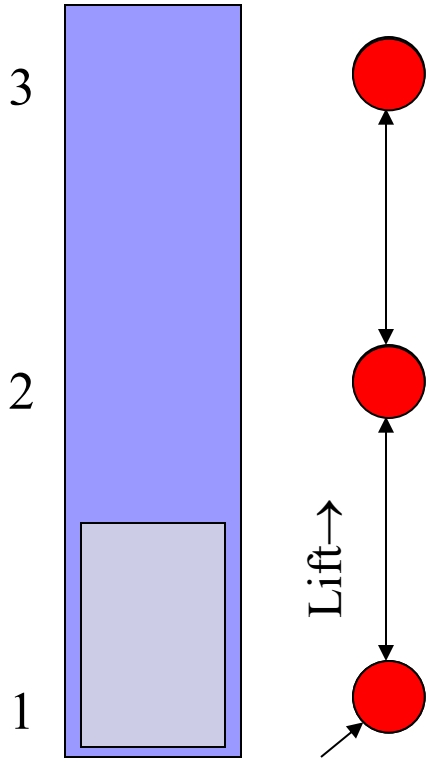
Warning: GHDL announces overflow 4/4




```
if state/=s0 then -- previous state
    predR<=State2uint(state_t'PRED(state));
end if;
if state/=s4 then -- previous state
    state:=state_t'SUCC(state);
end if;
```

Limits of the automatic model

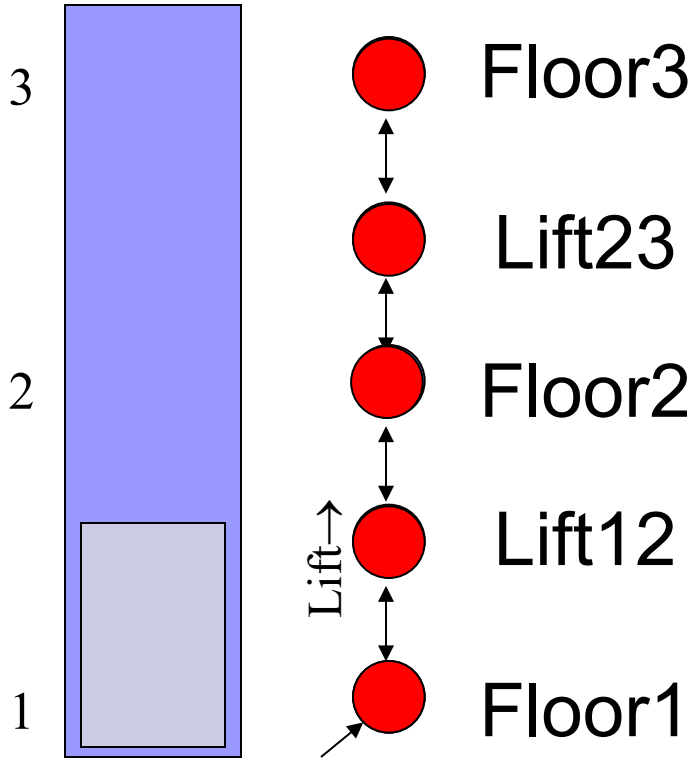
Movement of Lift by States



The state concept can also reduce continuous systems to the minimum information necessary for our task.

If we are interested only in a position on the floors,
Then, one elevator can be described in three states.

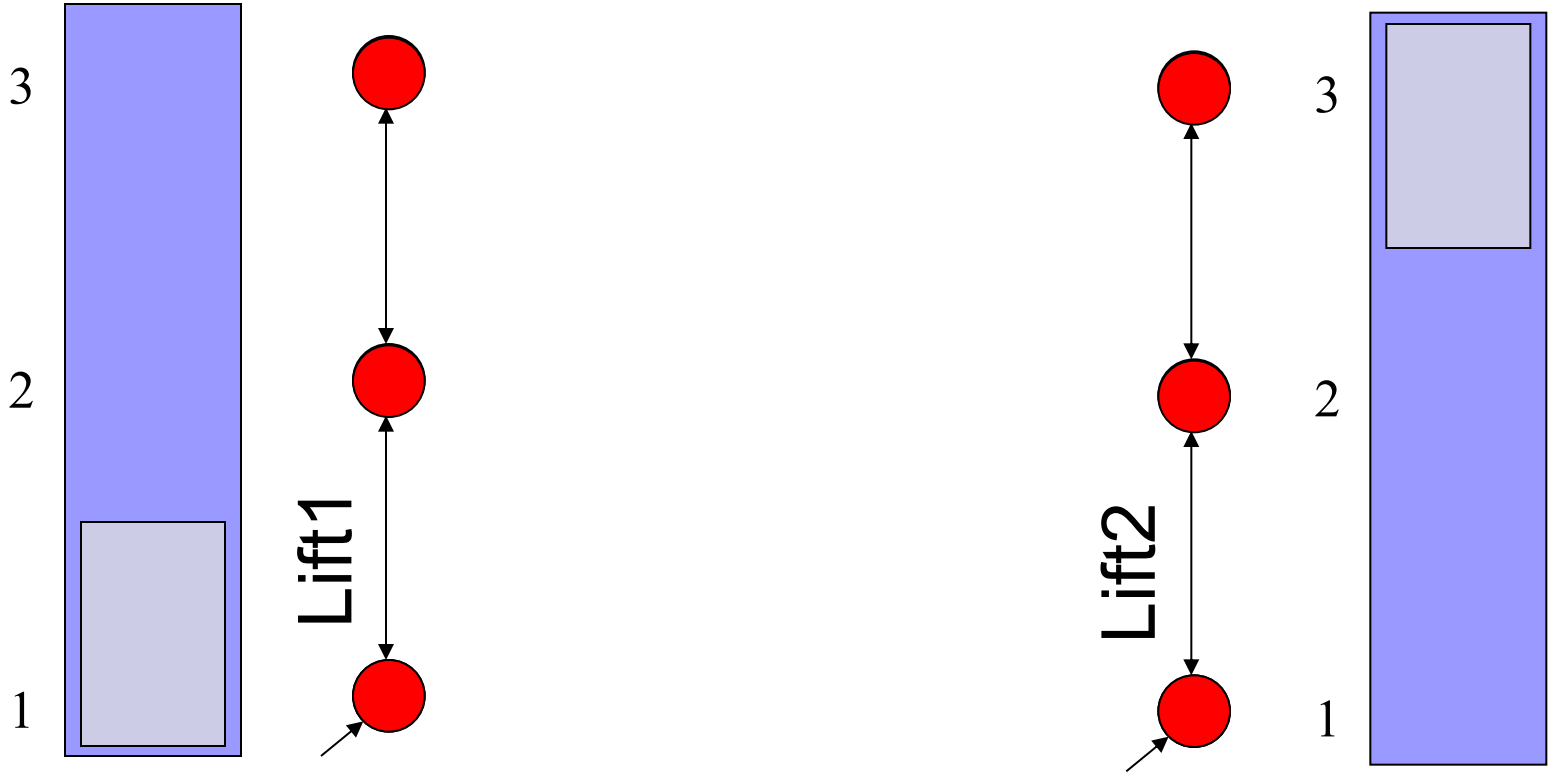
Movement of Lift by States



We choose the states based on real events, that we need.
The model can include any detail ...

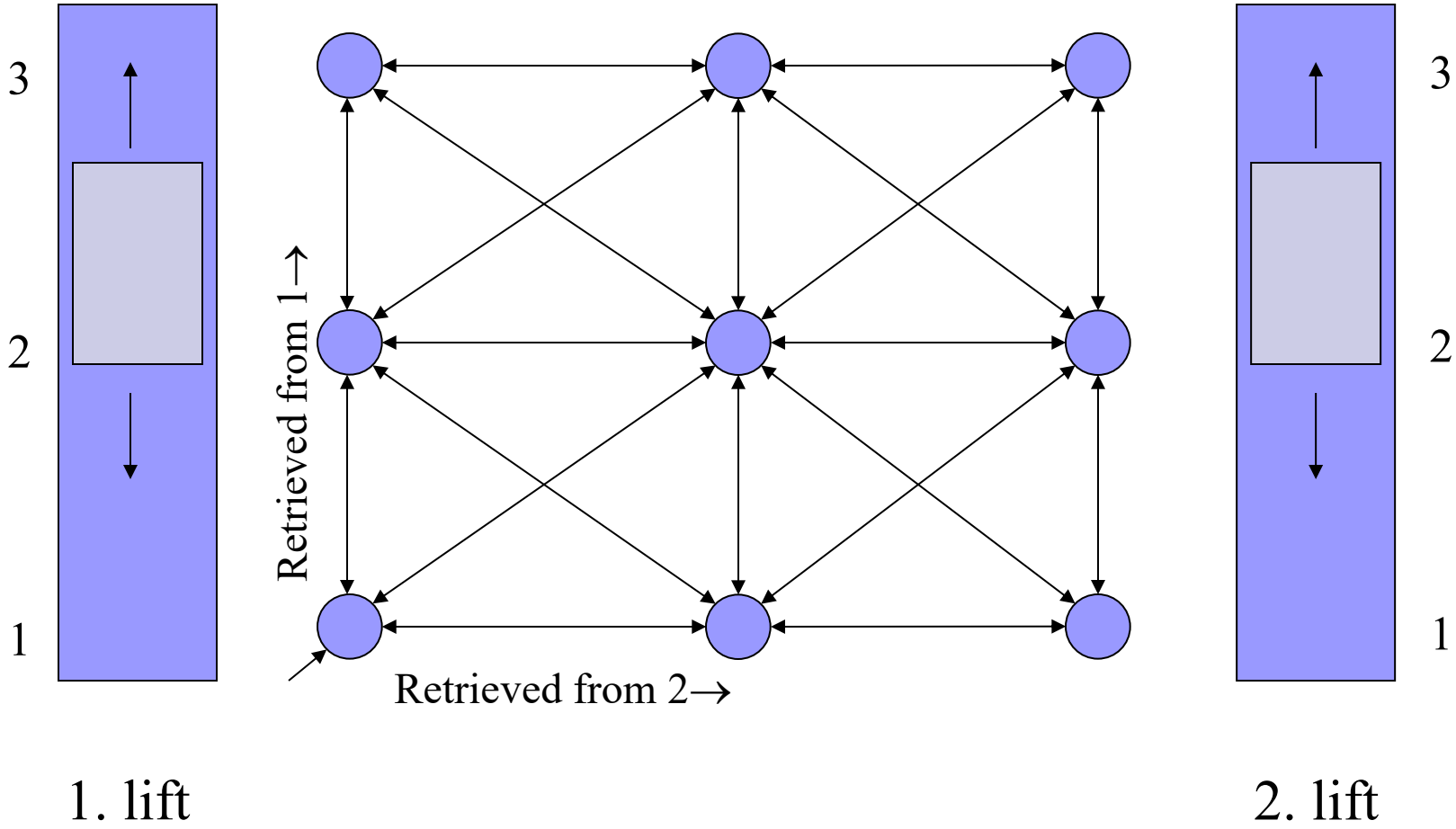
If we are interested not only in the floors, but in the lift ride, we add the necessary states.

Description of 2 Lifts by 2 FSM



Two lifts with floors can be described as a combination of 2 FSMs.

Model 2 lifts by 1 FSM



*The FSM is always only in **one state***
-> The number of states must include all possibilities!

Task

Sample D input on the rising and falling edges of C clock.

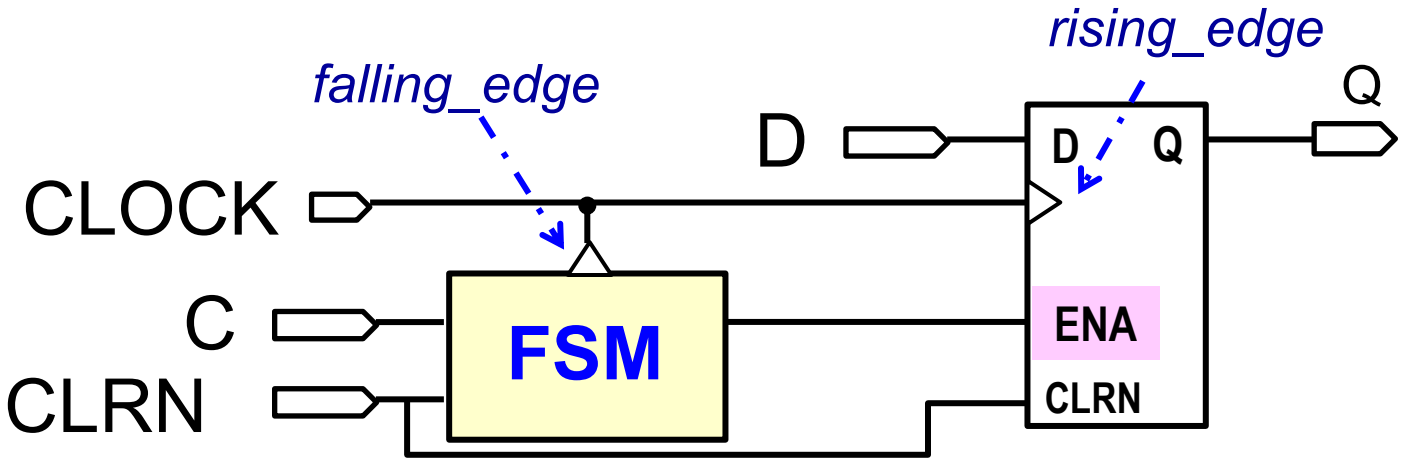
If it is solved by one FSM, we need 9 states.

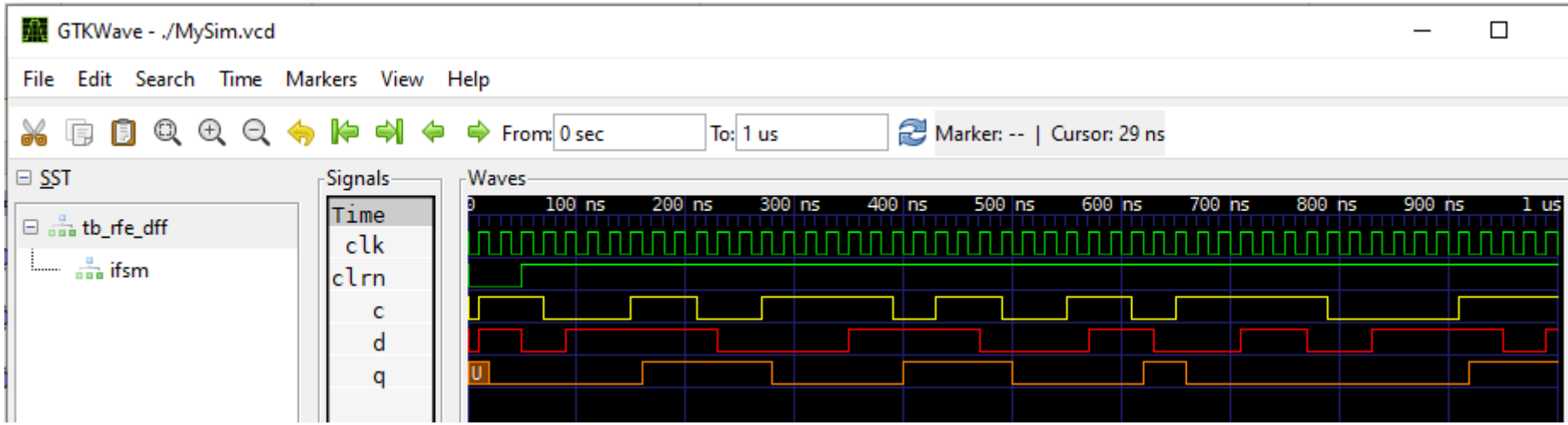
Why 9 states? How do we decompose?

- The number of 2 possible values of the C input is multiplied by the number of D values, so we have 4 possible states.
- Moore's FSM assigns **one output to each state**, so we need to distinguish if a state generates '0' and '1'; therefore, 4 states are doubled to 8 states. Oh, its initial diagram grows to a complex and non-transparent graph.
- But the requirement for '0' and '1' induces the possibility to split the solution into two parts. FSM will determine the state and the second circuit '0' and '1' value.
- In both cases, we must add one additional state for initialization, so our solution can consider up to 9 or 5 states.
- <u>We chose the decomposing</u>. One 5-state FSM will control a DFF flip-flop:
  - The FSM sets the ENA (enable) input of the DFF at both the rising and falling edges of input C, which we detect by comparing C with its past values.
  - Our FSM and the DFF must necessarily operate on the CLOCK:
    a) With a much higher frequency than changes of the C input.
    b) And they will utilize its opposite edges, e.g., the FSM uses the falling edge of the CLOCK and the DFF its rising edge to assure that the ENABLE input of the DFF is stable around its active CLOCK edge.

We decompose the task to 2 FSMs

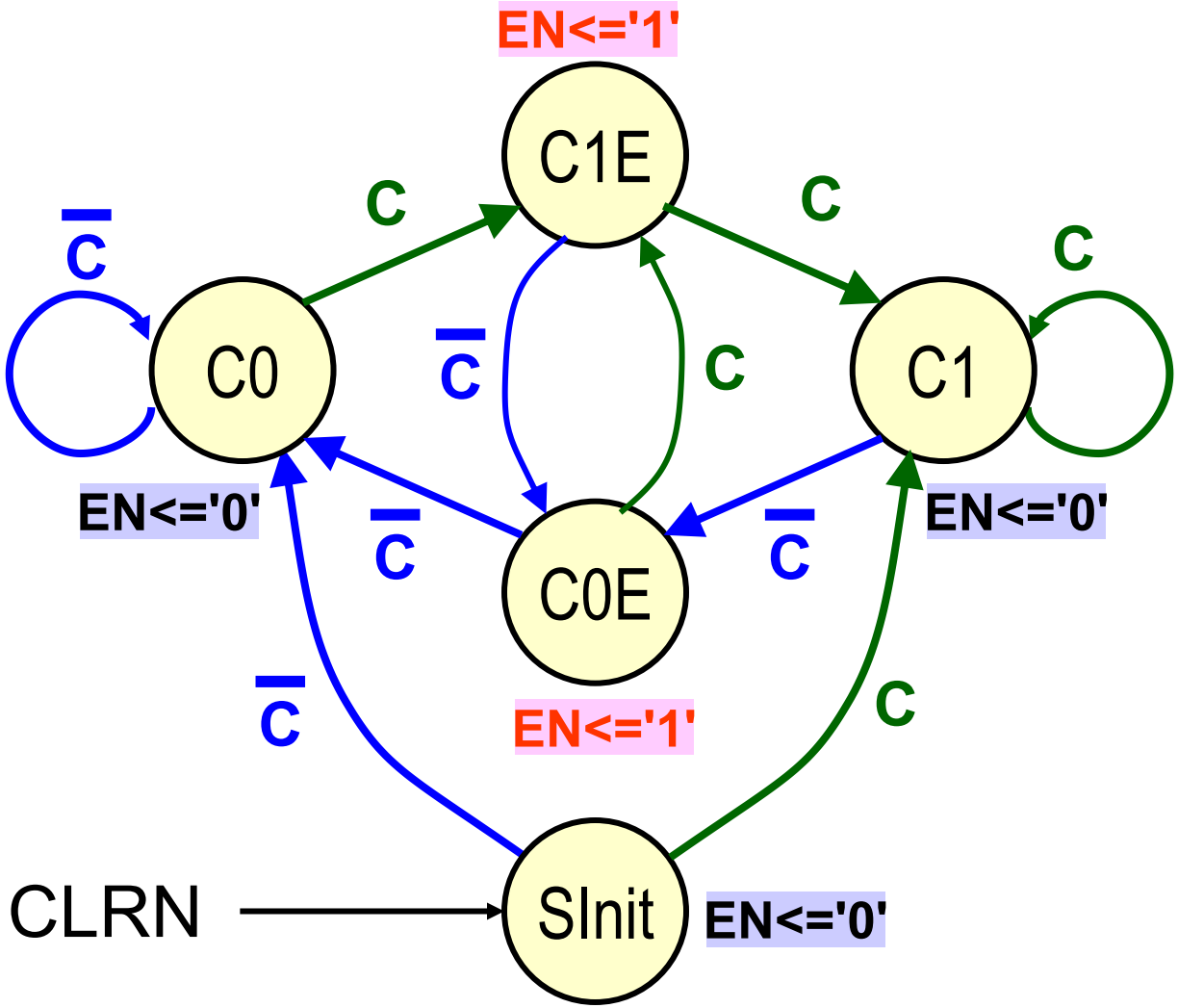
Let CLOCK be much faster than C changes, then the task can be simplified by decomposing it into an FSM controlling a DFF register.





Moore's FSM

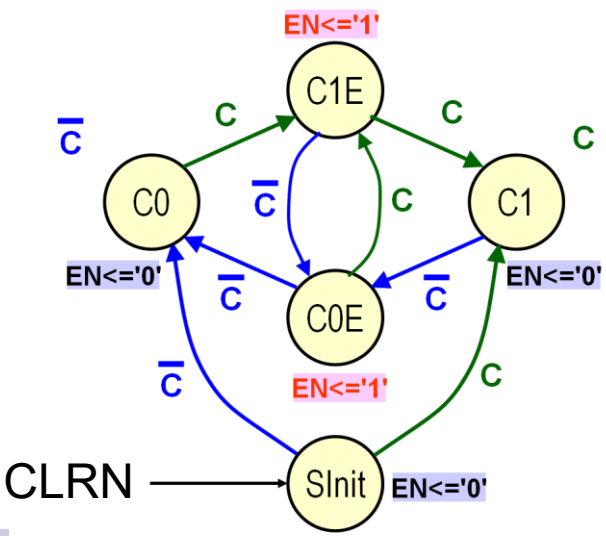
	C '0'	C '1'	EN
SInit	C0	C1	0
C0	C0	C1E	0
C1En	C0E	C1	**1**
C1	C0E	C1	0
C0En	C0	C1E	**1**



We cannot minimize it as a Moore automaton due to the absence of equivalent states with the same outputs. However, the EN output does not serve as a clock, and it can be created by logic of to a smaller Mealy FSM.

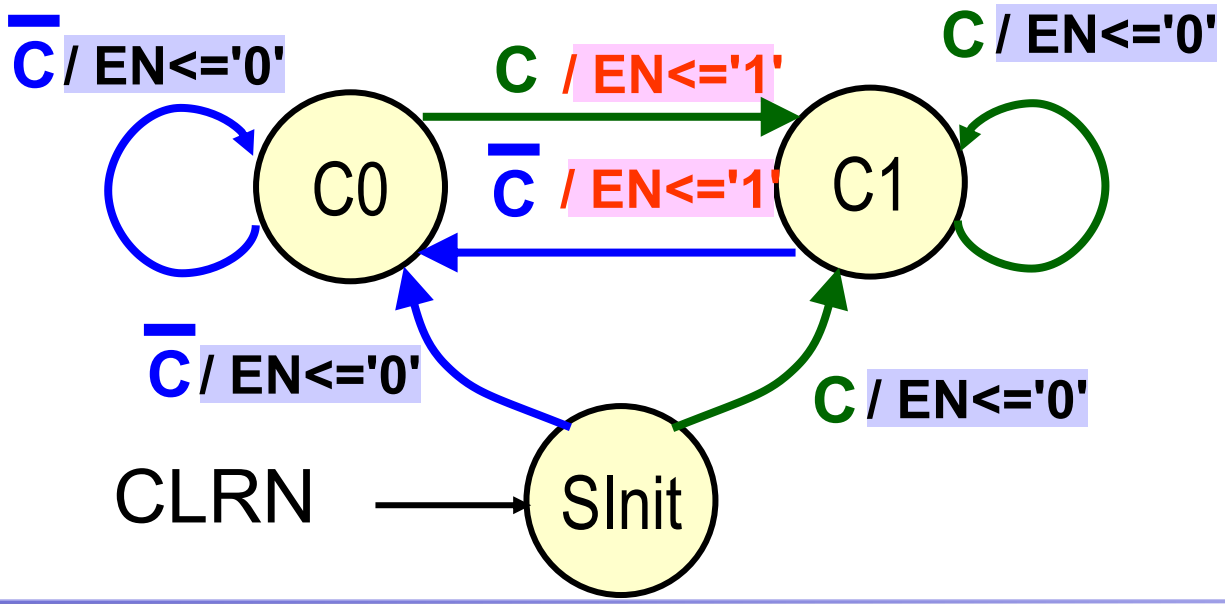
Moore FSM

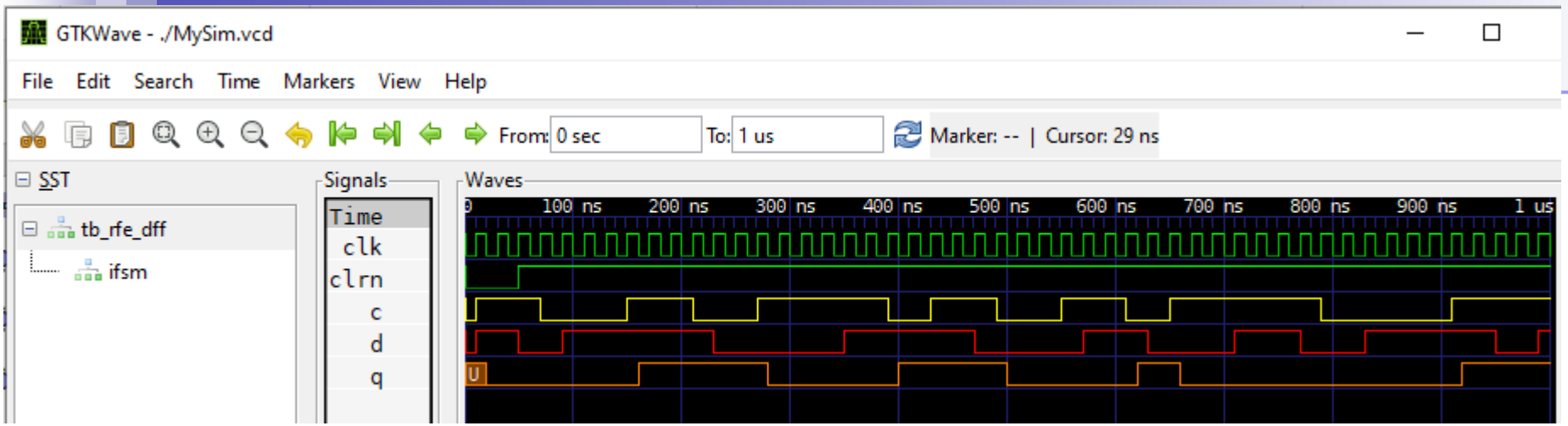
	C '0'	C '1'	EN
SInit	C0	C1	0
C0	C0	C1E	0
C1E	C0E	C1	**1**
C1	C0E	C1	0
C0E	C0	C1E	**1**

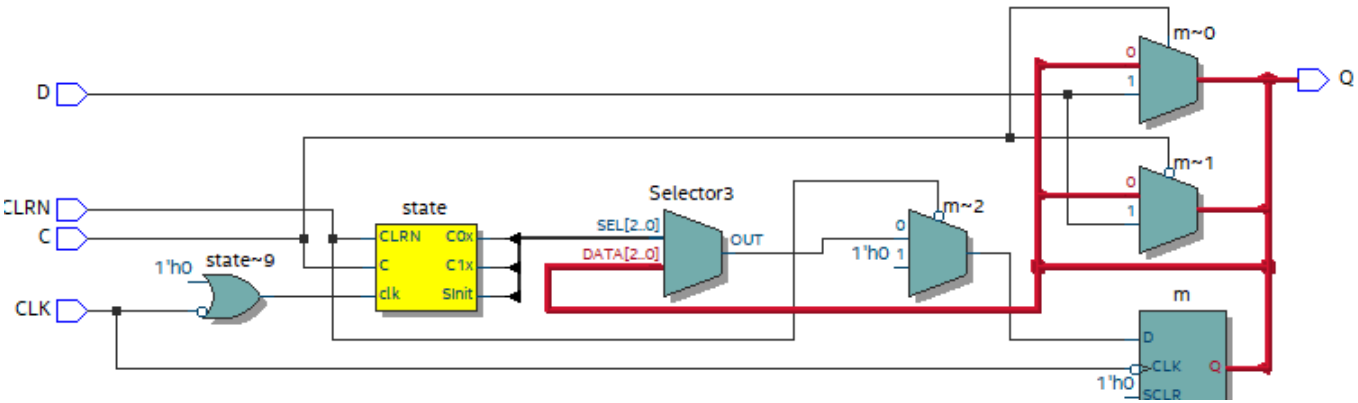


Smaller Mealyho FSM

			EN	
	C '0'	C '1'	C '0	C '1'
SInit	C0	C1	0	
C0x	C0x	C1x	0	**1**
C1x	C0x	C1x	**1**	**0**

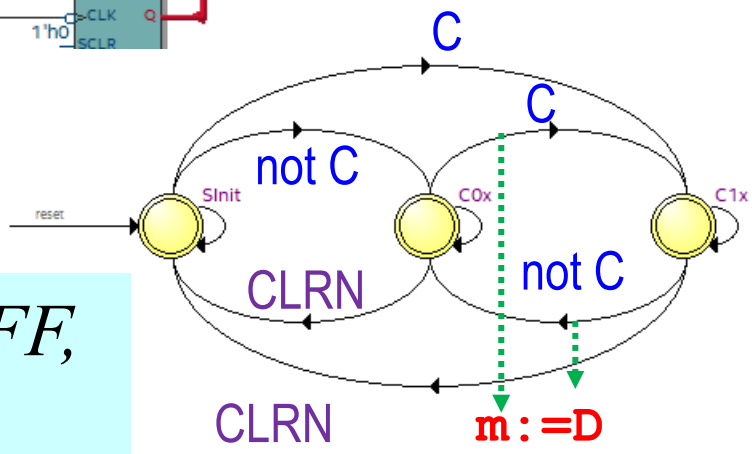






	Name	C1x	C0x	SInit
1	SInit	0	0	0
2	C0x	0	1	1
3	C1x	1	0	1

Transitions / Encoding



Quartus did not use enable for DFF, but created it by loading its Q.

The whole code

```
library ieee; use ieee.std_logic_1164.all;
entity RFE_DFF is    port ( CLK, C, D, CLRN : in std_logic:='0';      Q : out std_logic);
end entity;
architecture rtl of RFE_DFF is
begin
   process (CLK)
   type state_t is (SInit, C0x, C1x);
   variable state : state_t:=SInit;
   variable m:std_logic;
   begin
    iRedge : if falling_edge(CLK) then
      iClrn:if CLRN='0' then state:=SInit; m:='0';
      else
        case state is
          when SInit => if C='1' then state:= C1x; else state:=C0x; end if;
          when C0x =>  if C='1' then state:= C1x; m:=D; end if;
          when C1x =>  if C='0' then state:= C0x; m:=D; end if;
       end case;
      end if  iClrn;
    end if iRedge;
    Q<=m;
  end process;
end architecture;
```